mybibli

User Manual

The mybibli authors
https://github.com/guycorbaz/mybibli

Version 1.21.1 — October 31, 2026

Licensed under AGPL-3.0-or-later.
This manual covers mybibli version 1.3.1 and later (the v1.1.0 floor is still enforced for fresh installs — see the Installation chapter for the seed-gate background).

Contents

Preface 1

1 Installation 3
- 1.1 System requirements . . . 3
- 1.2 Pre-installation checklist . . . 4
- 1.3 Method 1 — Bundled database . . . 4
- 1.4 Method 2 — External database (Synology NAS example) . . . 6
- 1.5 Environment variable reference . . . 7
- 1.6 First boot — the setup wizard . . . 9
- 1.7 Verifying the install . . . 9

2 Configuration 10
- 2.1 Storage locations and hierarchy . . . 10
- 2.2 Spine labels: V-codes and L-codes . . . 11
- 2.3 Hardware: barcode scanner . . . 12
- 2.4 Reference data . . . 13
- 2.5 System settings . . . 14
- 2.6 Users . . . 15

3 Everyday use 16
- 3.1 The navigation bar . . . 16
- 3.2 The home page . . . 16
- 3.3 Cataloging a new title via barcode . . . 16
- 3.4 Searching the collection . . . 18
  - 3.4.1 Saved searches . . . 18
- 3.5 Management labels . . . 18
- 3.6 Moving a volume to another location . . . 19
- 3.7 Loans . . . 19
- 3.8 Shelf audit . . . 20
- 3.9 Shelving workflow . . . 21
  - 3.9.1 Undoing a scan action . . . 21
- 3.10 Wish list . . . 21
  - 3.10.1 Adding an entry . . . 22
  - 3.10.2 Browsing at the bookstore . . . 22
  - 3.10.3 Marking a book as bought . . . 22
  - 3.10.4 Auto-detect on catalog scan . . . 22
- 3.11 What the library is worth . . . 22

4 Multiple media types 24
- 4.1 Books . . . 24
- 4.2 Comics and BD . . . 24

4.3 Audio releases . . . 24
4.4 Films and series . . . 25
4.5 Picking the media type at scan time . . . 25

5 Metadata providers **26**
5.1 The provider chain . . . 26
5.2 API keys . . . 28
5.3 Where the keys live . . . 29
5.4 Provider health . . . 29
5.5 Rate limits and back-pressure . . . 29
5.6 Troubleshooting empty metadata . . . 30

6 Roles and permissions **31**
6.1 Who can do what . . . 31
6.2 Sessions and the inactivity timeout . . . 32
6.3 The last-active-admin guard . . . 32
6.4 Password reset . . . 32
6.5 Anonymous sessions . . . 33

7 Backup and restore **34**
7.1 What to back up . . . 34
7.2 Backup procedure . . . 34
7.3 Recommended cadence . . . 35
7.4 Restore procedure . . . 36
7.5 Migration to a new host . . . 36

8 Upgrade and migration **37**
8.1 Routine upgrades (1.y to 1.z) . . . 37
8.2 Skipping intermediate versions . . . 37
8.3 The 1.0.x to 1.1.0 jump . . . 38
8.4 Reading the release notes . . . 39
8.5 What's new in 1.21.1 . . . 39
8.6 What's new in 1.21.0 . . . 39
8.7 What's new in 1.20.1 . . . 40
8.8 What's new in 1.20.0 . . . 41
8.9 What's new in 1.19.0 . . . 41
8.10 What's new in 1.18.0 . . . 42
8.11 What's new in 1.17.0 . . . 43
8.12 What's new in 1.16.0 . . . 44
8.13 What's new in 1.15.0 . . . 44
8.14 What's new in 1.14.1 . . . 45
8.15 What's new in 1.14.0 . . . 45
8.16 What's new in 1.13.0 . . . 46
8.17 What's new in 1.12.0 . . . 47
8.18 What's new in 1.11.0 . . . 47
8.19 What's new in 1.10.0 . . . 47
8.20 What's new in 1.9.1 . . . 48
8.21 Older release notes . . . 48
8.22 Rollback . . . 48

9 Best practices **49**
9.1 Storage hierarchy . . . 49

9.2 V-code rolls . . . 50
9.3 Genre versus Dewey . . . 50
9.4 Audit cadence . . . 50
9.5 User accounts . . . 51
9.6 Loans . . . 51
9.7 Metadata providers . . . 51
9.8 Upgrades . . . 51

10 Troubleshooting **52**
10.1 Reading the logs . . . 52
10.2 App does not start . . . 52
10.3 Cannot log in . . . 53
10.4 Stray users in the `users` table after the wizard . . . 53
10.5 Metadata never resolves . . . 54
10.6 Search returns too little . . . 54
10.7 Cover images are missing . . . 55
10.8 Where to file bugs . . . 55

11 API & integrations **56**
11.1 Why an HTTP API? . . . 56
11.2 Creating an API key . . . 56
11.3 Authenticating . . . 57
11.4 Endpoints . . . 57
11.4.1 GET /api/v1/titles . . . 57
11.4.2 GET /api/v1/titles/{id} . . . 58
11.4.3 GET /api/v1/genres, /locations, /series . . . 58
11.4.4 GET /api/v1/dewey/{prefix} . . . 58
11.4.5 PATCH /api/v1/titles/{id} . . . 58
11.5 Examples . . . 59
11.5.1 `curl` . . . 59
11.5.2 Python . . . 59
11.5.3 An LLM classifier loop . . . 59
11.6 Revoking a key . . . 60
11.7 What the API does *not* expose . . . 60

12 Operations & debugging **61**
12.1 Where the logs live . . . 61
12.2 Tailing logs from the host . . . 61
12.3 Log levels . . . 62
12.4 Reading the structured JSON . . . 63
12.5 Sharing a log excerpt for a bug report . . . 63
12.6 Retention & rotation . . . 63
12.7 Resetting a forgotten administrator password . . . 64

13 AI agents and proposals **66**
13.1 What an agent can and cannot do . . . 66
13.2 Before you start: the network . . . 67
13.3 Step 1 — create a *propose* key . . . 67
13.4 Step 2 — allow the name the agent calls mybibli by . . . 67
13.5 Step 3 — connect the agent . . . 68
13.5.1 What any MCP client needs . . . 68
13.5.2 Claude Code . . . 68

13.5.3 Checking the endpoint with the MCP Inspector . . . 69
13.5.4 When the connection fails . . . 70
13.6 The tools . . . 70
13.7 Reviewing proposals . . . 71
13.7.1 Seeing that proposals are waiting . . . 71
13.7.2 The list . . . 71
13.7.3 The detail page . . . 72
13.7.4 Editing before you accept . . . 72
13.7.5 Accepting . . . 72
13.7.6 Outdated proposals . . . 73
13.7.7 Rejecting . . . 73
13.7.8 The resubmission delay . . . 73
13.8 Expected tomes and the wish list . . . 73
13.9 Proposed covers: the staging area . . . 74
13.10 Revoking an agent's key . . . 74

14 Glossary **75**

Index **78**

Preface

Who this manual is for

This manual is written for the person who installs, configures and runs a mybibli instance for their household, a small association, or a community library. It assumes you can:

- open a terminal and run `docker compose` commands;
- edit a plain-text configuration file with the editor of your choice;
- read a URL printed in a log line and open it in a browser.

No Rust, MariaDB or HTMX knowledge is required. The manual is deliberately scoped to the *operator* role — building mybibli from source or modifying its code is covered by `CLAUDE.md` and the planning artifacts in the repository.

Conventions

Commands. Shell snippets are shown in a fixed-width box:

```
docker compose up -d
```

Configuration files. File excerpts use the same style with the file name called out in the surrounding text. Comments inside the snippet use the file's native comment marker.

Callouts. Three sidebars highlight important text:

Note

A *note* provides background or context that helps you understand why something works the way it does. Skipping it is safe.

Tip

A *tip* suggests a practice that makes day-to-day use smoother. Try it once you are comfortable with the basics.

Warning

A *warning* flags a step where a mistake can lose data, lock you out of the admin panel, or expose the instance to unauthorized access. Read these in full.

Cross-references. Chapters and sections are linked in the PDF — click any blue title or page number to jump there. The index at the end maps every important term to its page of first use.

Getting help

The canonical source of truth for mybibli is the GitHub repository:

https://github.com/guycorbaz/mybibli

Open an issue against the `guycorbaz/mybibli` repository when you hit a bug, a documentation gap, or a missing feature. The issue templates ask for the version (visible at the bottom of any page in the admin panel), your install method, and the steps to reproduce.

Manual coverage

This manual covers mybibli **1.1.0 and later**. Older versions shipped a security-relevant bug where the first-launch setup wizard was silently bypassed — see chapter 8 for the upgrade path from 1.0.x.

Happy cataloging.

Chapter 1

Installation

This chapter walks you through installing mybibli on your own hardware. The reference deployment target is a home server, NAS, or any always-on Linux box that can run Docker.

1.1 System requirements

- **Operating system:** any Linux distribution, macOS, or Windows host capable of running Docker. The host kernel must support cgroups v2 (any distribution from 2021 onward).
- **Docker:** version 24 or later, with the `compose` plugin. Test with `docker compose version`.
- **Disk:** at least 1 GB free for the application image plus the database. Cover images add roughly 50 KB per title.
- **Memory:** 512 MB is enough for a household-sized library (a few thousand titles); 1 GB gives generous headroom for the metadata-fetch pipeline.
- **Network:** outbound HTTPS to the metadata providers you choose to use (see chapter 5). LAN-only deployments work without any inbound port forwarding.

Warning

Do not install mybibli versions below 1.1.0. Every published image with a tag below `1.1.0` contains a seed migration that creates the credentials `admin/admin` and `librarian/librarian` on *every* fresh install, including production. The first-launch setup wizard is silently bypassed. Use 1.1.0 or later; the pre-1.1.0 images will be removed from Docker Hub to prevent accidental use.

Warning

mybibli is not built to face the open internet. It is designed for one household on one local network, and several deliberate choices follow from that: there is no limit on login attempts, no second authentication factor, the session cookie carries no `Secure` attribute unless you set one (section 1.5), and the catalogue is readable without signing in — that last one is a feature, not an oversight.
Do not forward a port to it. If you need to reach your library from outside the house, two shapes are supported:

- **A private tunnel** — Tailscale, WireGuard or your router's VPN. Nothing is pub-

lished; your phone joins the network instead. This is the recommended shape, and the one the author runs.

- **A reverse proxy terminating TLS** — and then you must also set `MYBIBLI_COOKIE_SECURE=true`. That second half is the one people forget: without it the session cookie is still accepted over plain HTTP, and the proxy has bought you less than you think.

The reasoning behind each of these choices is written down in `docs/auth-threat-model.md` in the source repository, and `SECURITY.md` says how to report anything that contradicts it.

1.2 Pre-installation checklist

Before you start:

1. Pick a host directory for the mybibli files (e.g. `/srv/mybibli` on Linux, `/volume1/docker/mybibli` on Synology DSM).
2. Decide how cover images are stored. The shipped `docker-compose.yml` declares a Docker *named volume* (`mybibli_covers`) — zero configuration, Docker manages persistence and backup via `docker volume`. If you prefer a host-visible directory (Synology DSM file browser, manual rsync backups), pick a host path now (e.g. `/volume1/docker/mybibli/covers`) and you will switch the compose file to a bind mount in step 3 below.
3. Decide how logs are stored (v1.7.0+). The shipped `docker-compose.yml` declares a third named volume (`mybibli_logs`) for the daily-rotating log files. Like for covers, you can keep the Docker-managed volume (zero-config) or switch to a host-visible bind mount for shipping logs to DSM Log Center, journald, or an external log collector. See section 1.3.
4. Choose a strong password for the bundled MariaDB `root` and application users — or, for an external database, create a dedicated database and user up front (see section 1.4).

1.3 Method 1 — Bundled database

This is the simplest path. The shipped `docker-compose.yml` file declares two services: the mybibli application and a MariaDB instance pinned to a known version. Both are managed as one unit.

1. Get the compose file and `.env`

Download the `docker-compose.yml` and `.env.example` from the release you intend to install (or check out the repository at the matching tag):

```
mkdir -p /srv/mybibli && cd /srv/mybibli
curl -O https://raw.githubusercontent.com/guycorbaz/mybibli/v1.21.1/docker-compose.yml
curl -O https://raw.githubusercontent.com/guycorbaz/mybibli/v1.21.1/.env.example
cp .env.example .env
```

1b. (Optional) Switch covers to a bind mount

By default, the shipped `docker-compose.yml` stores downloaded cover images in the Docker-managed named volume `mybibli_covers`. This works zero-config and persists across container upgrades (issue #213). Most operators should leave it as-is.

If you want covers visible in your host file manager (Synology DSM File Station, manual rsync backups, etc.), edit the compose file:

- In the `mybibli` service's `volumes:` block, comment out the `- mybibli_covers:/app/covers` line and uncomment the `- ${COVERS_HOST_PATH:-./covers-host}:/app/covers` line right below it.
- In `.env`, set `COVERS_HOST_PATH` to the host path you picked in the pre-installation checklist (e.g. `COVERS_HOST_PATH=/volume1/docker/mybibli/covers`).
- Create the directory before starting the stack (`mkdir -p`) so Docker doesn't auto-create it owned by root.

1c. (Optional) Switch logs to a bind mount

New in **v1.7.0**. By default, mybibli writes its daily-rotating log files to the Docker-managed named volume `mybibli_logs` mounted at `/var/log/mybibli` inside the container. Files are named `mybibli.log.YYYY-MM-DD` and the in-process appender purges files older than 30 days automatically.

If you want logs visible directly on the host (Synology DSM Log Center, journald shipping, external log aggregator, manual `tail`), edit the compose file:

- In the `mybibli` service's `volumes:` block, comment out the `- mybibli_logs:/var/log/mybibli` line and uncomment the `- ${LOGS_HOST_PATH:-./logs-host}:/var/log/mybibli` line right below it.
- In `.env`, set `LOGS_HOST_PATH` to the host path you want (e.g. `LOGS_HOST_PATH=/volume1/docker/mybibli/log`
- Create the directory before starting the stack (`mkdir -p`) so Docker doesn't auto-create it owned by root.

The logs volume is forensic-only — you can wipe it at any time without affecting catalog data. See chapter 12 for log-tailing and log-level controls.

2. Edit `.env`

Open `.env` in your editor. Every variable carries an explanatory comment; the values that matter for a bundled-database install are:

```
DATABASE_URL=mysql://mybibli:mybibli_password@db:3306/mybibli?charset=utf8mb4
MYSQL_HOST=db
MYSQL_DATABASE=mybibli
MYSQL_USER=mybibli
MYSQL_PASSWORD=<choose-a-strong-password>
MYSQL_ROOT_PASSWORD=<choose-a-strong-password>

HOST_PORT=8080         # change if 8080 is already taken on the host
APP_LANGUAGE=en        # or 'fr' for the French UI by default

MYBIBLI_COOKIE_SECURE=false   # set to true only behind HTTPS
```

The `DATABASE_URL` *must* reference the same user, password and database name as the `MYSQL_*` values — the compose file rebuilds the URL from those parts but the Rust binary reads `DATABASE_URL` directly, so the two must agree.

Warning

The default passwords in `.env.example` (`mybibli_password`, `root_password`) are placeholders, not credentials to keep. Change them before the first `docker compose up`. After the database has been created, rotating these passwords requires also rewriting the MariaDB user entries — easier to get it right the first time.

3. Start the stack

```
docker compose up -d
docker compose logs -f mybibli
```

The first launch:

1. pulls the `gcorbaz/mybibli:1.1.0` (or later) image and the MariaDB image;
2. starts the database and waits for it to be healthy;
3. runs the mybibli database migrations (one-time schema setup);
4. starts the HTTP listener on the port you configured.

You can stop tailing the logs with `Ctrl-C`; the containers keep running detached.

4. Open the setup wizard

Visit `http://<host>:8080/setup` in a browser. The first-launch setup wizard appears (covered in section 1.6).

1.4 Method 2 — External database (Synology NAS example)

If you already run MariaDB on your NAS — for example via the Synology **MariaDB 10** package — you can point mybibli at it instead of running a second database in Docker. The mybibli image is unchanged; only the compose file and `.env` differ.

1. Create the database and user

Connect to the external MariaDB as `root` and run:

```
CREATE DATABASE mybibli CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'mybibli'@'%' IDENTIFIED BY 'choose-a-strong-password';
GRANT ALL PRIVILEGES ON mybibli.* TO 'mybibli'@'%';
FLUSH PRIVILEGES;
```

Note

On Synology DSM, MariaDB 10 listens on port `3307` by default (not 3306 — that port is reserved for the older MySQL package). Open **MariaDB 10 → Settings** and tick *Enable TCP/IP connection* so containers on the same Docker bridge can reach the server.

2. Trim the compose file

Edit `docker-compose.yml` and remove the bundled database:

- delete (or comment out) the entire `db:` service;
- delete (or comment out) the `depends_on:` block on the `mybibli` service;
- in the bottom `volumes:` block, remove `mybibli_db_data:` only — **keep** `mybibli_covers:`. Cover images are local to the app and must survive container upgrades regardless of where the database lives (issue #213).

3. Point `.env` at the external database

```
DATABASE_URL=mysql://mybibli:strong-password@host.docker.internal:3307/mybibli?charset=
    utf8mb4
```

Substitute the host name and port that match your network. From a mybibli container running on the same Docker bridge as the NAS host, `host.docker.internal` resolves to the host on macOS and recent Linux Docker installs; on older Docker engines, use the LAN IP of the NAS instead (e.g. `192.168.1.10`).

The `MYSQL_*` variables are now unused — leave them blank or delete them.

4. Start the application

```
docker compose up -d
docker compose logs -f mybibli
```

The first launch runs the schema migrations against the external database. The Synology MariaDB package will hold this schema for as long as the package exists; backing it up is covered by the NAS-level backup tools (see chapter 7).

Tip

The external-database setup is the recommended path when you already maintain a database server, because it gives you one backup pipeline to monitor instead of two. The bundled-database setup is faster to get going if you have no other workloads on the host.

1.5 Environment variable reference

The full reference lives in the file `.env.example` at the repository root — every variable carries an inline comment. The table below summarises which variable applies where.

All boolean variables follow a **strict accept-set**: only the strings `1`, `true` and `TRUE` count as "on". Anything else (including the empty string, `0`, and `false`) is treated as "off". This avoids the classic footgun where a stale shell value like `0` reads as "set" and silently flips an opt-out.

Variable	Default	Purpose
Database		
`DATABASE_URL`	— (required)	MariaDB connection string (DSN).
`MYSQL_HOST`	`db`	Hostname for the bundled DB service.
`MYSQL_PORT`	`3306`	Port for the bundled DB service.
`MYSQL_DATABASE`	`mybibli`	DB name for the bundled service.
`MYSQL_USER`	`mybibli`	DB user for the bundled service.
`MYSQL_PASSWORD`	—	DB user password for the bundled service.

Variable	Default	Purpose
MYSQL_ROOT_PASSWORD	—	Root password for the bundled service.
HTTP server		
HOST	0.0.0.0	Bind address inside the container.
PORT	8080	Bind port inside the container.
HOST_PORT	8080	Port published by Docker on the host.
Application		
MYBIBLI_LOG_LEVEL	info	v1.7.0+. Runtime log level. Plain level (trace/debug/info/warn/error) or an EnvFilter directive list. Takes precedence over RUST_LOG. Also flippable from /admin > System.
MYBIBLI_LOG_DIR	/var/log/mybibli	v1.7.0+. Directory for daily-rotating log files inside the container. Mapped to the mybibli_logs named volume.
LOGS_HOST_PATH	./logs-host	v1.7.0+. Optional bind-mount host path; used only when you uncomment the bind-mount line in docker-compose.yml.
RUST_LOG	mybibli=info	Legacy tracing filter. Honored when MYBIBLI_LOG_LEVEL is unset.
APP_LANGUAGE	en	Default UI language for anonymous visitors. v1.7.0 adds de + it to en / fr.
COVERS_DIR	./covers	Filesystem path for cover-image storage.
COVERS_HOST_PATH	./covers-host	Optional bind-mount host path; used only when you uncomment the bind-mount line in docker-compose.yml.
Hardening		
MYBIBLI_COOKIE_SECURE	false	Set to true only behind HTTPS.
CSP_REPORT_ONLY	false	Switch CSP header to report-only mode.
MYBIBLI_MCP_ALLOWED_HOSTS	loopback	Host values the AI-agent endpoint /api/mcp answers (comma-separated host or host:port). Default localhost,127.0.0.1,::1; a set value replaces it. See section 13.4.
MYBIBLI_MCP_ALLOWED_ORIGINS	—	Origin values accepted on /api/mcp; empty refuses any request carrying one.
Metadata providers (migrated to DB on first boot)		
GOOGLE_BOOKS_API_KEY	—	Optional key for Google Books.
OMDB_API_KEY	—	Optional key for OMDb (films/series).
TMDB_API_KEY	—	Optional key for TMDb (films/series).
Optional dev / test overrides		
MYBIBLI_SKIP_SETUP	false	Bypass the first-launch wizard. *Never on prod.*
MYBIBLI_SKIP_STARTUP_PURGE	false	Skip the boot-time auto-purge.
MYBIBLI_SEED_DEV_USERS	false	Seed dev admin/admin + librarian/librarian. *Never on prod.*

Provider base-URL overrides (BNF_API_BASE_URL, GOOGLE_BOOKS_API_BASE_URL, …) exist solely to point the metadata pipeline at the in-tree mock server during automated tests. Leave them unset in production.

1.6 First boot — the setup wizard

On a clean install, every request is redirected to `/setup` until the wizard completes. The wizard has four steps:

1. **Admin account.** Choose a username and a strong password for the first administrator. This step is the whole reason the wizard exists — once the account is created, the wizard's gate predicate flips and subsequent visitors land on the public catalog instead.
2. **Metadata providers.** Paste in any API keys you have on hand (Google Books, OMDb, TMDb). All keyed providers are optional — you can skip this step entirely and add keys later from `/admin > System > Metadata Providers`.
3. **Preferences.** Pick the default UI language for anonymous visitors and the loan overdue threshold (in days).
4. **Done.** A summary screen. After confirming, the wizard writes `settings.setup_completed_at` and disables itself for the lifetime of the installation.

The wizard is idempotent: if you close the browser between steps, reopen `/setup` and the wizard resumes server-side at the right step (it queries the database on every load — there is no client state to lose).

1.7 Verifying the install

After the wizard finishes:

- Log in at `http://<host>:8080/login` with the admin credentials you just created.
- Visit `/admin > Health`. The page should show non-zero entity counts (zero on a clean install — that is expected) and a green check next to each metadata provider that has been reachable in the last 5 minutes.
- Scan an ISBN (the search field at the top of the home page) to confirm the metadata pipeline resolves a real title. The first scan takes a couple of seconds; the result appears in the timeline of recent additions on the home page once the background fetch completes.

If anything goes wrong, jump to chapter 10.

Chapter 2

Configuration

Once the setup wizard is behind you, the day-one configuration work consists of three blocks: defining your *storage locations*, populating the *reference data* (genres, contributor roles, volume states, location types), and adjusting the system-wide *settings* (overdue threshold, auto-purge cadence, provider keys).

Everything in this chapter is configured from the `/admin` panel — there is no separate configuration file. The panel has six tabs:

- **Health** — read-only status dashboard. Besides versions, entity counts and provider status, it shows the last volume number (V-code) and last shelf number (L-code) ever printed, trashed items included, each with the next free number — the place to look before printing a new sheet of barcode labels.
- **Users** — create / deactivate users, change roles.
- **Reference data** — the taxonomy tables, and the management-label vocabulary (section 3.5).
- **Trash** — view, restore and permanently delete soft-deleted items.
- **System** — global settings.
- **API keys** — mint and revoke the keys the HTTP API authenticates with (chapter 11), and the *propose* keys of AI agents (chapter 13).

Only the `Admin` role sees the panel; `Librarian` and `Anonymous` users get a 403 from every `/admin/*` route.

2.1 Storage locations and hierarchy

Storage locations are the physical spots where your volumes live: a room, a shelf, a row, a numbered box, a specific cell in a cabinet. mybibli organises them as a **tree** — every location has at most one parent. The hierarchy is arbitrary: you decide both the levels and the labels.

A workable household hierarchy might be three levels deep:

```
Living room          (room)
   Bookshelf A      (shelf)
        Row 1       (row)
        Row 2       (row)
   Bookshelf B      (shelf)
```

```
        Row 1       (row)
Bedroom              (room)
    Nightstand     (shelf)
```

Three levels is enough to find any volume in seconds without forcing the hierarchy to mirror every nook of your living space. A flat list (one big "shelf" per physical shelf) works equally well for collections under a few hundred volumes.

Editing the tree

Open `/locations`. The tree is rendered as a nested list with inline create / rename / move / delete affordances. Drag-and-drop is *not* supported (mybibli ships with a CSP-strict no-JS policy); moves are done by editing a node and picking a new parent from a dropdown.

Location node types

Each location node carries a **node type** — "room", "shelf", "row", "box", anything you like. Node types are managed under `/admin > Reference data > Location node types`. They are pure labels; mybibli does not enforce that a "shelf" must sit under a "room".

Tip

Renaming a node type cascades to every location that uses it (the type is stored by name in the locations table). Deleting a node type is refused as long as one location still references it.

2.2 Spine labels: V-codes and L-codes

Note

Two things are called "labels" in mybibli. This section is about the *spine labels* you print and stick on a book or a shelf — the barcodes that make scanning work. The other kind, added in 1.18.0, is the *management label* you put on a record to say something about its state ("To check", "Damaged binding"); those live in section 3.5. Nothing is printed for them, and they never leave the application.

mybibli prints two kinds of barcode labels:

- **V-codes** — one per *volume* (the physical book on a shelf). Format: `V` followed by a zero-padded sequence (e.g. `V0042`).
- **L-codes** — one per *storage location*. Format: `L` followed by a zero-padded sequence (e.g. `L0007`).

V-codes and L-codes are scanned with the same barcode scanner you use for ISBNs at cataloging time. The scanner's prefix (`V` or `L`) tells mybibli which lookup to perform.

Printing labels

Open `/locations` or the relevant volume page and click **Print label**. mybibli renders a printable HTML page with the Code-128 barcode and human-readable text. Print on plain paper or sticker stock — the encoding is forgiving enough that a low-cost inkjet works for V-codes and L-codes alike.

Tip

Print V-code stickers *before* you start cataloging in bulk. A roll of pre-printed V0001 through V0500 means you can grab a sticker, stick it on the back cover, and scan in one motion — no context-switch between the cataloging UI and the printer.

2.3 Hardware: barcode scanner

mybibli is designed around a barcode scanner but does not strictly require one — every ISBN, V-code and L-code field accepts manual keyboard input as a fallback. That said, for any collection beyond a handful of titles, a cheap USB scanner pays for itself within the first dozen scans.

What to buy

Any *HID-keyboard-mode* USB scanner in the 15–30 € range works. The HID mode is what makes the scanner driver-free: the OS sees it as a regular keyboard and the scanned digits land directly in whatever input field has focus. Almost every general-purpose USB scanner on the market ships in this mode by default; look for the words "HID", "keyboard emulation" or "plug-and-play" on the product page.

The scanner must support the two symbologies mybibli uses:

- **EAN-13** — the 13-digit code on the back cover of books, CDs, DVDs, and most consumer products. Universally supported.
- **Code-128** — the symbology mybibli uses to encode V-codes and L-codes on the labels it prints (see section 2.2). Universally supported.

You do *not* need a 2D imager (the kind that reads QR codes) — a basic 1D laser or LED scanner is enough. mybibli does not use QR codes.

Configuration

Three settings on the scanner matter. Most scanners ship them correctly out of the box; if your scans behave strangely, check the scanner's manual or the configuration sheet of barcodes that comes in the box:

- **Terminator / suffix:** `Enter` (also called `CR` or `Carriage Return`). mybibli's cataloging form auto-submits on Enter, so this turns each scan into one round-trip. Anything else (`Tab`, no terminator) will not work without manual key presses.
- **Prefix:** none. Some scanners can prepend a fixed string before the payload — leave it disabled.
- **Keyboard layout:** *must* match the layout of the host the browser runs on. If your laptop uses an AZERTY layout but the scanner is configured for QWERTY, the digits come out as letters (e.g. `1234567890` arrives as `&é"'(-è_çà`) and no ISBN ever resolves. The scanner's manual will have a code chart to switch.

Tip

USB scanners on a Synology NAS host: it does not matter that the scanner is plugged into the laptop you use to access the mybibli web UI rather than the NAS itself. The

scanner is a keyboard for the browser, not for the server. The same setup works with a phone-side scanner app, an industrial Bluetooth scanner paired to the laptop, or any USB stick scanner.

Confirmed-working hardware

Because mybibli only needs a keyboard-emulation scanner and a browser, the device you run it on can be a desktop, a laptop or a tablet. Combinations confirmed working in the field:

- **iPad (Safari) + Eyoyo barcode reader.** A tablet at the cataloging station with a wireless Eyoyo scanner — handy for walking the shelves during an audit.
- **Linux workstation + Inateck BCST-36.** A fixed cataloging desk with a Bluetooth Inateck BCST-36 scanner.

These are examples, not a compatibility whitelist — any scanner in HID keyboard mode (see *What to buy* above) works the same way.

What to do when the scanner is unavailable

Every scan field also accepts hand-typed input. Click into the field labelled “Scan ISBN” (or “Scan V-code” / “Scan L-code” elsewhere), type the digits, press Enter. The form behaves identically.

2.4 Reference data

Four taxonomy tables drive the dropdowns you see while cataloging. They live under `/admin > Reference data`:

Genres The genres assigned to titles. mybibli ships with a baseline FR/EN seed (Fiction, Non-fiction, Biography, …) — edit or extend to match your collection.

Contributor roles Author, Illustrator, Translator, Editor, Narrator, …Used by title detail pages to attribute the right people to the right slots.

Volume states The physical condition of an exemplaire — New, Good, Worn, Damaged, …Useful for loan returns (mark a volume Worn when the borrower damaged it).

Location node types See section 2.1 above.

Labels Your working vocabulary — ”To check”, ”Re-read”, ”Damaged binding”, whatever your practice calls for. Unlike the four above, one label applies to *both* titles and volumes, and an item can carry several. They are visible to librarians only: a passing visitor sees nothing of them, anywhere.

CRUD semantics

All four tables share the same CRUD pattern:

- **Create** adds a row. If the name collides with a soft-deleted row of the same table, the existing row is reactivated and the result reads *Reactivated* instead of *Created*.
- **Rename** updates the name. For location node types, the new name cascades to every location that uses the type.

- **Delete** performs a soft-delete *only when the row is unused*. If any title still references the genre / role / state / type, the deletion is refused with a count of dependents.

Reference data is not translated

Reference data values are stored verbatim — the FR seed creates a genre called "Roman" and that is the literal text English-speaking users will see if they switch the UI to English. Only the UI chrome (button labels, navigation, error messages) is translated.

2.5 System settings

The `/admin > System` tab groups every global setting. The panel is divided into sections, each with its own save button:

Loans

- **Overdue threshold (days).** Defines how many days a loan can run before it surfaces as overdue on the dashboard. Default 30.
- **Auto-purge interval (seconds).** How often the background scheduler hard-deletes items older than 30 days in the trash. Default 86400 s (24 hours).

Sessions

Since 1.11.0 the **session inactivity timeout** is set here, in *hours* (1–720, default 4). It controls two things at once: how long a signed-in session survives without activity, and the lifetime of the session cookie in the browser — so a tablet that locks its screen mid-cataloguing session no longer loses the login when the browser discards session-only cookies. Applied on the next request; already-open sessions are not cut off retroactively.

Metadata providers

One row per keyed provider (Google Books, OMDb, TMDb). Each row has a key input and a *Clear* button. Saving a key takes effect on the next metadata fetch — no restart, no re-deploy.

Note

If the same provider key is set in `.env` *and* cleared from the UI, the next reboot re-migrates the value from `.env`. The operator's deployment-time intent wins on boot. To make a Clear durable across reboots, also remove the variable from `.env` (or `docker-compose.yml`).

The same section also carries the lookup **timeouts** (since 1.7.9): the per-provider metadata timeout (how long the lookup chain waits for one provider before falling through to the next, default 5 s) and the provider-health probe timeout (default 10 s), both bounded 1–60 s. Since 1.9.0 the metadata timeout can additionally be **overridden per provider**: the form lists every registered provider with its own field; leave a field empty to use the global default. Useful when one provider (say, a slow national library) needs more headroom without slowing the whole chain down. Changes apply on the next scan — no restart.

Since 1.11.0 the same form also carries the **bulk cover-refetch delay** (milliseconds, 0–60000, default 1000): the pause between titles when you run *Re-fetch missing covers* from the Health tab. Too fast a pace makes providers throttle the whole run — Google Books answers request bursts with errors and no covers come back. The default adds about two minutes per hundred titles, which is irrelevant for a background action. The run now also retries a throttled title

with increasing back-off before giving up on it, and finishes with a summary on the Health tab: covers recovered, provider errors, and titles for which no cover is available anywhere.

Since 1.13.0 the Health tab carries a second bulk action next to the cover refetch: **Backfill metadata from libraries**. It re-runs the metadata lookup for *every* cataloged title that has a lookup code — not just the cover-less ones — to fill the UNIMARC-aligned fields introduced in 1.13.0 on titles cataloged before the upgrade. Since 1.14.0 it queries both national libraries, the BnF and the Library of Congress, which extends its reach to English-language books. It only ever fills gaps: a value already present, or one you edited by hand, is never overwritten. Both bulk actions share the same pacing delay, throttle back-off and single-run lock, so only one of them can run at a time.

Proposals

The **delay before a rejected proposal can be resubmitted**, in days (0–3650, default 90). While it runs, an AI agent cannot file a proposal equivalent to one a librarian rejected — same kind and same subject, whichever key files it. Applied to the next proposal, no restart. See section 13.7.8.

Language

Sets the default UI language for *anonymous* visitors. Authenticated users override this with the language toggle in the navigation bar — the choice is stored on their user row.

2.6 Users

Open `/admin > Users` to manage user accounts. Each row shows the username, role, active flag, and last-seen timestamp. The available actions are:

- **Create user** (top of the page) — username, password, role.
- **Edit** — change role or rename. Cannot change another user's password from this view; password resets go through the user's own `Account` page.
- **Deactivate** — soft-delete the user. Their sessions are wiped immediately; they cannot log back in.
- **Reactivate** — undo a deactivation (from the `Trash` tab).

Two safety guards apply:

- **Self-deactivate is blocked.** You cannot deactivate the account you are logged in as.
- **Last-active-admin guard.** The system always keeps at least one active admin. Deactivating the last one is refused with a 409 conflict and a clear error.

The three roles (Anonymous, Librarian, Admin) are detailed in chapter 6.

Chapter 3

Everyday use

This chapter walks through the workflows you will run dozens of times once the library is live: cataloging a new volume, searching your collection, moving a volume, lending it out, taking it back, and auditing what is on the shelves.

3.1 The navigation bar

The navigation bar stays at the top of the page while you scroll. On a window at least 1280 px wide it shows every entry in one row; below that width the entries move into a menu opened by the *hamburger* button (three horizontal lines) at the right of the bar, which also holds the *Log out* button — and, on a phone, the *Log in* link. Librarians and Admins find a **Proposals** entry there, with a badge counting the proposals waiting for a decision (chapter 13).

3.2 The home page

The home page is the dashboard. From top to bottom:

- **Scan field.** A focused input that accepts an ISBN, a V-code or an L-code. The scanner-guard layer routes the input to the right page depending on its prefix.
- **Indicators row.** Recent additions, overdue loans, series with gaps, unshelved volumes — each click jumps to the filtered list.
- **Recent activity.** The last seven days of cataloging and returns.

3.3 Cataloging a new title via barcode

1. Focus the home-page scan field (it is selected by default on page load). Scan the ISBN / EAN-13 of the back cover.
2. mybibli redirects to the catalog page, which displays a skeleton title with the barcode you scanned. A background task fires off requests against the metadata provider chain.
3. Within a few seconds, the skeleton fills in: title, contributors, publisher, year, cover image. A small indicator confirms which provider resolved the title.
4. Pick a **location** from the dropdown (or scan the L-code of the destination shelf) and pick a **volume state** (typically *New*). Click **Add volume**.

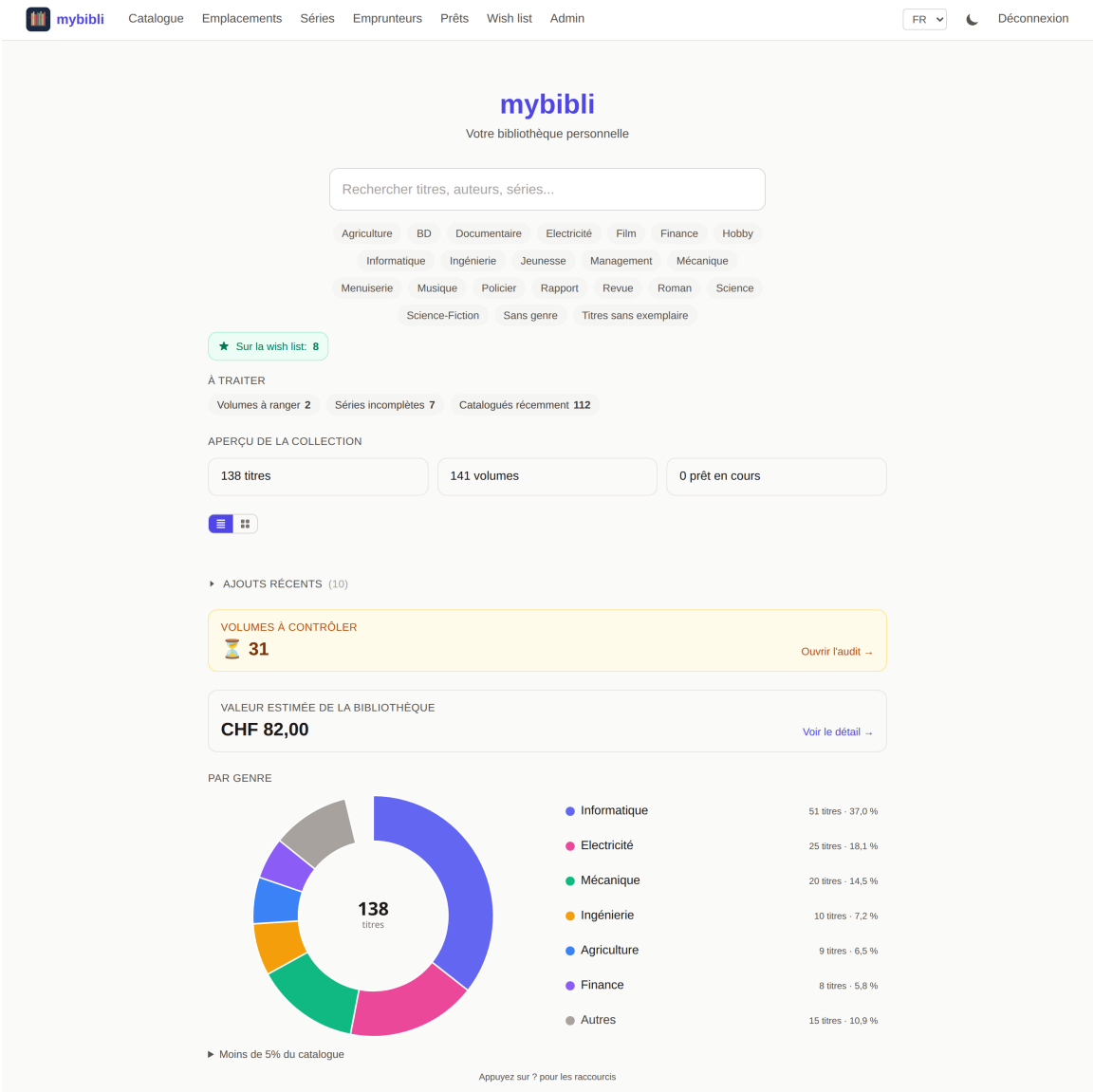


Figure 3.1: The home page on a live install: search bar, genre filter chips, “À traiter” counters, “Aperçu de la collection” totals, and a recent-additions strip with cover thumbnails fetched through the metadata-provider chain.

5. Print and stick a V-code label on the volume.

If the metadata pipeline fails to resolve the ISBN, the title appears with an *Unresolved* badge. You can edit the title fields manually and the volume is still created.

Tip

Cataloging in batches works best when the V-code sticker roll, the scanner and the printer are all within arm’s reach. Most of the time is spent scanning, not waiting for mybibli — the metadata fetch is asynchronous, so you do not have to wait for it to finish before moving on to the next volume.

Printing fresh label sheets

Before printing a new sheet of V-code or L-code labels, check the small line at the top of the catalog page (since 1.12.0, Librarian and above): *"Labels in use up to V0142 · L0037 — print new label sheets starting after these numbers."* The numbers include items currently in the trash — a sticker that was ever printed and stuck on a book is never proposed again, even if that volume was deleted since.

3.4 Searching the collection

The `/search` page accepts:

- full or partial **title**;
- **contributor name** (matches on author by default; other contributor roles are scored lower);
- **ISBN / EAN-13** — exact match;
- **V-code** or **L-code** — exact match;
- **Dewey code** (for libraries that use Dewey).

Results are sorted by relevance. Click a result to open the title page; click a contributor to filter the catalog by their work; click a location label to see every volume currently in that location.

Tip

You can "search" a single volume by scanning its V-code from any page that has the home-page scan field — that includes the home page, `/search` and `/loans`. Faster than typing.

3.4.1 Saved searches

Recurring lookups ("comics without a cover", "uncategorized titles", "everything overdue") can be saved once and re-run with a single click. On the home page, compose the search you want — type a query, click a filter chip, choose a sort order — then open the **Saved searches** dropdown next to the sort control and use *Save current search* to name and store it.

Saved searches are **shared across the whole instance**: every librarian sees and can run the same set. To run one, open the dropdown and click its name — the home page reloads with that query, filter and sort applied. Each entry also offers *Rename* and *Delete* (both confirm in a modal). Deleting is reversible: saving a new search with the same name restores the old one.

Note

Saved searches capture the existing browse criteria (query, one filter chip, sort column and direction) — they are not a free-form query builder. The control is visible to librarians and administrators only.

3.5 Management labels

A genre says what a book *is*. A condition state says how a copy has *aged*. Neither says "I am not sure this record is right, come back to it" — and that sentence is the one a working catalogue

needs most. Since 1.18.0, *management labels* carry it.

They are not the spine labels of section 2.2: nothing is printed, nothing is scanned, and an anonymous visitor never sees them. They exist for the person maintaining the library.

Defining the vocabulary

An admin creates the vocabulary once, in *Administration* → *Reference data* → *Labels*: "To check", "Re-read", "Damaged binding", "Lent informally" — whatever your practice calls for. Labels are renamed and deleted from the same place, and like every other reference value they are never translated: what you type is what everyone sees, in any interface language.

Keep the list short. A vocabulary of six labels is used; a vocabulary of thirty is a second genre taxonomy nobody maintains.

Putting a label on something

A librarian applies labels from the *Labels* section of a title page or of a single volume page, choosing from that one shared vocabulary. The distinction matters:

- on a **title**, the label is about the record — "the metadata here looks wrong";
- on a **volume**, it is about that physical copy — "this one has a damaged binding", which says nothing about the other two copies on the shelf.

Finding what carries a label

`/labels` lists the vocabulary with a count against each entry. Opening one shows what carries it, in two sections — titles above volumes — each row linking through to its detail page. That is the whole point: a label you cannot find your way back from is a note in a drawer.

Note

Labels are internal. The handlers do not even run the queries for an anonymous visitor, so nothing leaks through the public catalogue — including the fact that a label exists.

3.6 Moving a volume to another location

1. Open the volume page (search by V-code or click through from a title page).
2. Click **Move**.
3. Scan the L-code of the destination location, or pick it from the dropdown.
4. Confirm.

mybibli records the move in the volume's location history so you can trace where a book has lived over time.

3.7 Loans

Lending a volume

1. Open `/loans` and click **New loan**.

2. Scan or pick the **borrower**. If the borrower does not exist yet, click *Create borrower* inline.
3. Scan or pick the **volume** (V-code).
4. Optionally set a custom **due date**; otherwise mybibli derives it from the overdue threshold (chapter 2).
5. Confirm.

The volume's current location is recorded on the loan row so a later return can restore it to the right shelf.

Returning a volume

1. Open `/loans` and find the active loan (filter on *Active* or scan the V-code).
2. Click **Return**.
3. Pick the post-return state (*Good*, *Worn*, …).
4. Confirm. The volume snaps back to its pre-loan location automatically.

Overdue loans

Loans that exceed the overdue threshold turn red on the `/loans` page and on the home-page *Overdue* indicator. You can run `/loans?filter=overdue` for a focused list.

3.8 Shelf audit

An *audit* is the routine "walk the shelves and confirm everything is where the database says it is". Since 1.7.x mybibli carries the workflow rather than leaving you to improvise it around the search page.

The unit of work is a flag on a volume: *to check*. You raise the flag, walk the shelf, and lower it as each copy is confirmed.

Raising the flag

- **One volume** — on its detail page, *Mark to check*. Useful when a reader mentions that something looks wrong, or when a scan surfaced a volume where it should not have been.
- **A whole shelf** — on a storage location's page, *Mark all volumes for audit*. This is how a planned pass starts: flag the shelf on Sunday evening, walk it during the week.

A flagged volume carries a *To check* chip wherever it is listed, so the flag is visible without opening the audit view.

Walking the shelf

The home dashboard shows a *Volumes to check* indicator with a count and a link; `/audit` is the list itself, sorted by location then V-code — the order you physically walk in. Each row carries a *Checked* button: press it as you confirm the copy in your hand, and the row leaves the list.

Two situations resolve themselves as you go:

- A volume on the shelf that the list does not expect — scan it; mybibli says it belongs elsewhere and you move it (section 3.6).
- A volume the list expects and the shelf does not hold — leave the flag raised. An unfinished audit list is a to-do list, not a failure; it is the only record of what you could not find.

Clear all flags empties the list in one action. Use it when you abandon a pass, not to finish one — a flag cleared without the volume in your hand records something you did not verify.

Tip

A quarterly audit catches misplaced volumes early — the longer a book has been on the wrong shelf, the harder it is to remember where it should be.

3.9 Shelving workflow

After a returns batch, the volumes pile up near the scanner station. mybibli's *shelving* flow speeds up putting them back:

1. Pick up a volume and scan its V-code.
2. mybibli shows the volume page with the original location prominently displayed.
3. Walk the volume to that shelf. Scan the L-code on the shelf to confirm.
4. Repeat.

Confirming with the L-code is optional but recommended — it leaves a fresh entry in the volume's location history that the next audit can rely on.

3.9.1 Undoing a scan action

Barcode work is fast, and the occasional mis-scan happens — a volume shelved at the wrong location, or a batch location activated by mistake. When you shelve a volume or activate a batch location on the catalog page, the feedback message shows an **Undo** button. Click it within **30 seconds** and mybibli reverses that last action:

- Undoing a shelving puts the volume back at its previous location (or leaves it unshelved if it had no location before).
- Undoing a batch-location activation restores the previously active location.

Only the *most recent* action can be undone, and the button disappears once the 30-second window closes. Undo only reverses the location change — it never deletes a title or volume you just created. If the window has passed, move the volume the usual way (section 3.6).

3.10 Wish list

The *wish list* is a separate list of books you'd like to buy, kept distinct from the owned catalog. It lives under `/wishlist` (see the navigation bar) and is available to Librarians and Admins. A missing tome of a series can be sent to it in one click from the series page (section 13.8).

3.10.1 Adding an entry

The *Add to wish list* form has two modes:

- **By ISBN.** Scan or type a 13-digit ISBN. mybibli looks the book up through the same metadata provider chain the catalog uses, then shows a preview card (title, author, publisher, year, cover). Click *Add to wish list* to save. Cover images come from the resolved provider's URL directly (e.g. Open Library Covers or Google Books); downloading the cover locally is on the roadmap.
- **Free-form.** Use this when you don't have an ISBN handy (a self-published title, a recommendation jotted down at lunch). Only *Title* is required; author, publisher and notes are optional.

3.10.2 Browsing at the bookstore

The list view collapses into a mobile-card stack on narrow viewports — designed to scan quickly with one thumb at a bookstore. Each card shows the title, the author (if known), the ISBN (if any), the publisher, and the date you added it.

To print a paper copy or save a PDF for offline use, click *Print* at the top of the list. mybibli renders a clean print-friendly page; use the browser's *Print to PDF* (File → Print → Save as PDF, or Ctrl/Cmd-P) to generate a PDF you can take to the bookstore.

3.10.3 Marking a book as bought

When you buy a book, open its wish list entry (or tap *Bought* directly from the list row), confirm in the modal, and the entry soft-deletes. The action is reversible from the Admin Trash panel for 30 days.

3.10.4 Auto-detect on catalog scan

When you catalog a freshly bought book through the standard scan flow (`/catalog`), mybibli checks whether its ISBN matches an active wish list entry. If it does, an emerald banner appears in the feedback area: *This ISBN is on your wish list — Remove it?* One click on the banner's *Remove from wish list* button opens the same *Mark as bought* modal, and confirming removes the entry. No silent deletions.

3.11 What the library is worth

Two optional amounts can be recorded against each volume: what it *cost* you (purchase price) and what it is *worth* now (current value), each with its own currency. Both are blank by default and nothing forces you to fill them — a library that is not an investment can ignore this section entirely.

If you do record them, `/stats/value` sums them across the catalogue: a total per currency, then a breakdown by genre and by series. Volumes with no value set are skipped rather than counted as zero, so a partially-valued catalogue reports the truth about the part you have valued rather than a diluted average.

- The **default currency** for new valuations is set in *Administration → System*; it is only a pre-fill, and any volume can carry another.
- The **home-dashboard card** showing the estimated total is *off* by default. Turn it on in the same place if you want the figure in front of you every day — some households do not.

Note

Insurance is the usual reason to keep these figures. If that is yours, record the purchase price as well as the current value: the first is a fact you can evidence, the second an estimate you will be asked to justify.

Chapter 4

Multiple media types

mybibli is not just for books. Four media types are first-class citizens:

- **Book** — the default. Single-volume novels, essays, reference works, children's picture books.
- **Comic / BD** — single-issue comics or multi-position omnibus albums (e.g. a "Tintin anthology" that physically gathers volumes 1–5).
- **Audio** — CDs, vinyl records, audiobooks on physical media.
- **Film / series** — DVDs, Blu-ray boxsets, TV-series collections.

The media type drives three things:

1. **Which metadata provider is consulted first.** ISBNs route to BnF + Google Books + Open Library; films route to TMDb + OMDb; audio routes to MusicBrainz.
2. **Which fields are exposed on the title page.** Audio titles surface a track list; films surface director, runtime, age rating.
3. **The icon shown next to the title** in search results and the catalog list.

4.1 Books

The default media type. Catalog as described in section 3.3. Multi-volume series (encyclopaedias, trilogies, …) are modeled with a *Series* entity in chapter 2.

4.2 Comics and BD

A single **omnibus** volume can contain several positions in a series. mybibli supports this with the *multi-position* feature: at volume creation time, you can set the volume's *position range* (for example, "contains volumes 5–7"). Series-gap detection (chapter 3) understands these ranges and will not flag positions 5, 6 or 7 as missing once the omnibus is on the shelf.

4.3 Audio releases

Audio titles use MusicBrainz as the primary metadata source. The barcode on the back of a CD (EAN-13) resolves the same way as an ISBN.

- **Tracks.** MusicBrainz returns the track list; mybibli renders it on the title page.
- **Format.** CD vs vinyl vs cassette is captured on the volume row (not on the title) — two identical pressings on different media are two volumes of the same title.

4.4 Films and series

Films and TV series use OMDb and TMDb as the primary metadata sources. A series boxset is modeled as a *title* with a multi-position volume covering the disc range.

Tip

OMDb is free for low volumes (under 1000 requests per day); TMDb is free with attribution. Either covers a household-sized DVD / Blu-ray shelf. See chapter 5 for setup.

4.5 Picking the media type at scan time

After you scan an unfamiliar barcode, mybibli guesses the media type from the EAN prefix (the GS1 country prefix and the resolved provider's response). The guess is shown on the catalog page; you can override it before clicking **Add volume** if it is wrong. Re-typing a title later is a few clicks (`/title/<id>`) and the provider chain re-runs against the new type if you trigger *Re-fetch metadata*.

Chapter 5

Metadata providers

When you scan an ISBN or an EAN-13, mybibli does not look the title up in a local catalog — it queries one or more external *metadata providers* until one returns a clean answer. This chapter explains the chain, where to get API keys, and how to rotate them safely.

5.1 The provider chain

mybibli walks the providers in the order best-suited to the guessed media type:

Provider	Media types	Notes	Key?
BnF	Books	French Bibliothèque nationale catalogue; best for French-language titles.	no
Library of Congress	Books	United States national library catalogue; the anglophone counterpart to the BnF. Also supplies MARC 21 zones.	no
K10plus	Books	German union catalogue. Zone completion only — it never resolves a title on its own. Consulted for ISBN prefixes 978-0/1/2/3.	no
Open Library	Books	Volunteer catalogue; broad English / multilingual coverage.	no
Google Books	Books	Commercial; good for trade non-fiction.	optional
MusicBrainz	Audio	Volunteer music metadata; very deep on CDs / LPs.	no
OMDb	Films, series	Free tier; rate-limited.	required
TMDb	Films, series	Free with attribution.	required
BDGest	Comics, BD	FR-language comic catalogue (web-scraped).	no

The first provider to return a *clean* record wins. "Clean" means: title present, at least one

contributor, and a year. Partial matches are recorded but mybibli keeps walking the chain until it finds a clean one or runs out of providers.

Bibliographic zones: the three catalogues

Six detailed cataloguing fields — statement of responsibility, edition statement, collection title and number, general note, original title — come only from the *national libraries*. No other provider carries them.

The two catalogues are complementary rather than redundant: the BnF describes the French-language corpus, the Library of Congress the English-language one. A title held by neither will have none of these zones, and a French title the BnF could not describe will not be rescued by the Library of Congress.

A third catalogue joined them in 1.16.0: **K10plus**, the German union catalogue. It behaves differently from the other two — it never resolves a title, it only completes zones on a record another provider already resolved — and on this catalogue it turned out to be the most productive of the three, filling zones on titles neither national library carried. It is consulted only for ISBN prefixes 978-0, 978-1, 978-2 and 978-3: the anglophone leftovers it exists for, its own German-language home ground, and the French prefix the BnF had already failed on. Any other prefix is skipped without a request, which keeps the one-request-per-second budget for the calls that can plausibly answer.

When the provider that resolved a title fills only some of the six zones, mybibli asks the others to complete the ones left empty. The rule is *first source wins*: a value already present is never overwritten. Only these six zones are merged — the title, authors and cover always come from a single provider.

None of this needs a key or any configuration.

The *Backfill metadata from libraries* button on the Health tab applies the same treatment to titles already in the catalogue.

Cover-image fallbacks

When the resolving provider returns metadata but no cover URL (BnF never carries image URLs in its UNIMARC payload; Google Books occasionally lacks `imageLinks` for self-published titles) mybibli tries three cover-only sources in order, keyed by ISBN:

1. The *BnF Couvertures* service (`https://api.bnf.fr`) — the Bibliothèque nationale de France's legal-deposit cover scans. The strongest source for French and Swiss-French publishers, typically at excellent image quality. Cover images are reused under the BnF metadata license (source attribution).
2. *Inventaire.io* (`https://inventaire.io`) — a libre, community-driven book database; particularly strong on French-language comics and manga.
3. The *Open Library Covers API* (`https://covers.openlibrary.org/`) — broad English / multilingual coverage.

The first source that has an image wins; the fallbacks run only when (a) the chain succeeded *and* (b) the resolving provider gave no `cover_url`, so a title for which no source has a cover anywhere will land cover-less without retry storms.

When no provider has a cover: upload your own

Some titles will never receive a cover from the automatic chain. In practice this is dominated by French, Swiss, and German trade and academic publishers (Dunod, Eyrolles, Slatkine, Mardaga, Kana, Casterman, and the like): Google Books returns nothing for their ISBNs and the Open Library Covers index does not hold them either. This is a structural gap in the free data sources, not a bug — no amount of re-fetching will fill it.

For those titles, **manual upload is the realistic path**. On a title's detail page a Librarian (or Admin) sees an *Upload cover* button under the cover area; titles with no cover get a more prominent call-to-action and a one-line reminder of this gap. Accepted formats are JPG, PNG, WebP and GIF, up to 10 MiB. The image is resized and stored locally, and a manually uploaded cover is protected from being overwritten by a later automatic re-fetch.

To find the titles that still need a cover, use the *Titles without cover* filter on the home page (Librarian and above), or follow the *Review titles without a cover* link shown on the Admin page's Health tab after a bulk cover re-fetch.

5.2 API keys

Three providers require an API key:

Google Books

Optional but raises the daily request budget significantly. Without a key the Google Books provider stays disabled — the chain still works (BnF → Open Library are key-less), but English-language trade non-fiction that BnF misses will land with empty metadata.

The Google Cloud Console flow trips up many first-timers because the API-key creation step is buried two menus deep. Walk through this exactly:

1. Open `https://console.cloud.google.com/` and sign in.
2. Top-bar project picker (next to the *Google Cloud* logo) → *New project* → name it anything (e.g. `mybibli-metadata`) → *Create*. Wait a few seconds for the project to be selected.
3. Open the hamburger menu (top-left, three horizontal lines) → *APIs & Services* → *Library*. Search for **Books API**, click it, then *Enable*.
4. Still under *APIs & Services*, open *Credentials*. Click *+ Create credentials* → *API key*. A modal pops up with a freshly generated key — copy it now.
5. Recommended: click *Restrict key* → under *API restrictions* pick *Restrict key* → tick only *Books API*. Leave *Application restrictions* on *None* (mybibli does not send a Referer header). Save.
6. In mybibli: `/admin > System > Metadata Providers` → paste into the **Google Books API key** field → *Save*. The next scan picks it up — no restart needed.

To verify the key is active, scan an English-only ISBN that BnF will not have (*Effective Java*, 9780134685991, is a good canary). The metadata should populate within a few seconds. If it stays empty, see *Troubleshooting* at the end of the chapter.

OMDb

1. Visit `https://www.omdbapi.com/apikey.aspx`.

2. Pick the free tier (1000 requests/day).
3. Confirm via the activation email you receive.
4. Paste the key into mybibli.

TMDb

1. Sign up at https://www.themoviedb.org/.
2. Open *Settings* → *API* and request a key for *Personal / non-commercial use*.
3. Paste the key into mybibli.

5.3 Where the keys live

Keys travel through two layers:

1. **Environment variables** (`GOOGLE_BOOKS_API_KEY`, `OMDB_API_KEY`, `TMDB_API_KEY`). These are migrated *once* on boot into the `settings` table — they are essentially a deployment-time bootstrap.
2. **The `settings` table**, exposed through `/admin > System > Metadata Providers`. This is the live source of truth at request time. Rotating a key from the UI takes effect on the very next metadata fetch.

The implication is asymmetrical:

- If you set a key only in the environment, the first boot copies it into the database and afterwards the database wins.
- If you set a key only in the UI, the environment is never consulted again.
- If you set both and then *Clear* the key from the UI, the database value goes empty — but on the next boot, the environment value is migrated back in. To make the UI clear durable, also remove the environment variable from `.env` / `docker-compose.yml`.

5.4 Provider health

`/admin > Health` pings every registered provider every five minutes and reports their status as a green check or a red cross. A red cross does not always mean the provider is down — it can also mean the API key is missing or invalid. The Health tab includes the last-checked timestamp and the HTTP status code for diagnostics.

5.5 Rate limits and back-pressure

mybibli applies a small per-provider rate limit (a couple of requests per second). Bulk-cataloging a hundred volumes in a row will not trigger remote 429 responses. If a provider does return 429, the request is logged and the next provider in the chain is consulted instead — the catalog flow never blocks waiting for a rate-limited provider.

Warning

Never commit a real API key to git. The `.env` file is in `.gitignore` for this reason; `.env.example` carries only placeholders. If a key leaks, rotate it at the provider console immediately — pasting a new key into mybibli takes effect within seconds.

5.6 Troubleshooting empty metadata

If a freshly scanned ISBN ends up with empty title / author / description fields, **ask the application first**. Open the title and press *Re-download metadata*: the lookup runs while you wait, and if it comes back empty the message underneath now says what actually happened rather than only that it failed. It distinguishes three situations that look identical from the outside:

- **"*N* sources were searched and none holds this code"** — the honest miss. No key, no retry and no waiting will change it; the book is not described in the catalogues mybibli consults. Fill the fields in by hand.
- **"*Source* could not answer just now"** — the source was busy or too slow. Genuinely worth pressing the button again in a few minutes.
- **"*Source* was not consulted: no API key is set"** — it was never asked. One field in *Administration* → *System* fixes it permanently.

The message names the sources, and the names are never translated: they are the libraries' own.

If you would rather reason it out yourself, walk this list:

1. **Provider status in `/admin > Health`.** If Google Books or Open Library show a red cross, those providers will be skipped on every fetch. Re-check the last-checked HTTP code — `403` or `401` usually means a missing or invalid API key, not a remote outage.
2. **Language of the title.** BnF is the primary provider and is FR-leaning. For English-only books, the chain falls through to Google Books — which means the key (see above) is the unblocker.
3. **ISBN format.** mybibli normalises EAN-13 vs ISBN-10 internally, but some providers index only one of the two. A second scan a few minutes later sometimes hits a different fallback path.
4. **Manually edit and save.** You can always edit title, genre, contributors etc. by hand from the title's detail page — saving works even when no provider returned anything. The genre `Non classé` is the default for any title that came in without classification data.

If the fields are still empty after configuring the right keys and waiting a few seconds, open an issue on the project repo with the failing ISBN — the maintainers add provider mocks for reproducible cases.

Chapter 6

Roles and permissions

mybibli has three user roles ordered by privilege:

1. **Anonymous** — anyone who lands on the site without a session cookie. Read-only access to the catalog. Cannot loan, edit or delete anything.
2. **Librarian** — authenticated user whose role is `librarian`. Day-to-day cataloging, loans, returns, location moves. Cannot manage users or change system settings.
3. **Admin** — authenticated user whose role is `admin`. Everything a Librarian can do, plus the `/admin` panel: users, reference data, trash, system settings.

6.1 Who can do what

Action	Anon	Lib	Admin
Browse / search the catalog	yes	yes	yes
View title and volume pages	yes	yes	yes
View location tree	yes	yes	yes
Scan / catalog a new title	no	yes	yes
Add or remove a volume	no	yes	yes
Move a volume between locations	no	yes	yes
Edit a title's metadata	no	yes	yes
Create / edit borrowers	no	yes	yes
Register / return loans	no	yes	yes
Edit storage location tree	no	yes	yes
See a series' expected tomes	yes	yes	yes
Manage expected tomes	no	yes	yes
Decide AI-agent proposals	no	yes	yes
Open `/admin`	no	no	yes
Create / deactivate users	no	no	yes
Edit reference data	no	no	yes
Restore from trash	no	no	yes
Permanently delete from trash	no	no	yes
Edit system settings	no	no	yes
Create / revoke API keys	no	no	yes

6.2 Sessions and the inactivity timeout

After login, mybibli sets a session cookie that travels with every subsequent request. The cookie is:

- `HttpOnly` — not exposed to JavaScript;
- `SameSite=Lax` — sent on top-level same-site navigations but not on cross-site requests;
- `Secure` (only when `MYBIBLI_COOKIE_SECURE` is set to `true` — see chapter 1).

The session has an **inactivity timeout** (default 30 minutes, configurable from `/admin > System > Loans`). Every authenticated request resets the countdown; if no request lands within the timeout window, the session is wiped server-side and the next request redirects to the login page.

A small *keep-alive toast* appears in the corner of the screen roughly two minutes before expiry — click it to extend the session without leaving your current page.

6.3 The last-active-admin guard

mybibli refuses to leave the installation without any active admin. The guard applies to two actions:

- **Deactivating a user.** If the user is the only active admin, the request returns a 409 conflict and the deactivation is refused.
- **Permanently deleting a user from the trash.** Same rule.

Self-deactivation is also blocked unconditionally (you cannot deactivate the account you are currently logged in as).

Warning

The guard counts only *active* admins. If you mass-deactivate admins and the guard fires, undo by reactivating someone from the trash tab. There is no recovery path through the wizard or the database without manual SQL surgery.

6.4 Password reset

mybibli does not implement password-reset emails — household / small-community deployments rarely benefit from the email plumbing. An admin can reset another user's password by:

1. Open `/admin > Users`.
2. Pick the user, click **Deactivate**.
3. Click **Reactivate** from the trash tab.
4. Re-create the user with a fresh password.

A user who knows their own password can change it from the `/account` page.

6.5 Anonymous sessions

Anonymous visitors also get a session row (used for CSRF protection and pending HTMX updates). Anonymous sessions expire after seven days of inactivity and are garbage-collected by a daily background task — they do not affect logged-in users.

Chapter 7

Backup and restore

A self-hosted library has no cloud safety net. This chapter covers what to back up, how often, and how to restore.

7.1 What to back up

Three artifacts together form a complete mybibli backup:

1. **The MariaDB database.** Holds every title, volume, location, contributor, loan, user, audit log and setting. Lose this and you start from zero.
2. **Cover images.** Downloaded during cataloging and persisted in the Docker volume `mybibli_covers` (or in your host bind-mount path if you switched per section 1.3). Lose this and the catalog still works, but every title page shows a generic placeholder until you re-fetch metadata title-by-title (issue #213 — pre-1.1.4 installs without this volume had covers disappear on every container upgrade). The same volume holds, in a hidden `.staging` folder, the covers proposed by AI agents and not yet decided (section 13.9).
3. **The `.env` file.** Holds passwords (database, API keys not yet migrated). Lose this and a re-deploy needs you to re-enter every credential.

The Docker images themselves are *not* backup-critical — you can always pull them again from Docker Hub.

7.2 Backup procedure

Database — bundled MariaDB

From the host running the compose stack:

```
docker compose exec -T db mariadb-dump \
    -u root -p"$MYSQL_ROOT_PASSWORD" \
    --single-transaction --routines --triggers \
    mybibli \
    > backups/mybibli-$(date +%Y%m%d-%H%M%S).sql
```

The `--single-transaction` flag is important: it dumps a consistent snapshot without locking the table for the duration of the dump. `--routines --triggers` preserves any stored routines (mybibli does not currently use any, but the flag is cheap and future-proof).

Compress the dump if disk space is a concern:

```
gzip backups/mybibli-*.sql
```

Database — external MariaDB

Run `mariadb-dump` directly against the external server. Synology DSM users can also schedule a *Hyper Backup* task that targets the MariaDB package — it produces a binary backup that restores faster than a SQL dump.

Covers and `.env`

For a host-mounted covers directory (bind-mount installs), a plain `tar` archive is enough:

```
tar czf backups/mybibli-files-$(date +%Y%m%d).tar.gz \
    .env "$COVERS_HOST_PATH"
```

For the default install (named Docker volume), back up the volume through Docker — the host path of a named volume varies by Docker version and is not stable, so don't try to `tar` it directly:

```
docker run --rm \
    -v mybibli_covers:/source:ro \
    -v "$(pwd)/backups:/backup" \
    alpine \
    tar czf "/backup/mybibli-covers-$(date +%Y%m%d).tar.gz" -C /source .
tar czf "backups/mybibli-env-$(date +%Y%m%d).tar.gz" .env
```

The throwaway `alpine` container mounts the named volume read-only, the host `backups/` directory writable, and writes the tarball. Same shape works for restoring — see the Restore section below.

7.3 Recommended cadence

- **Daily** — automated database dump. Even on a seldom-changed catalog, a daily dump means worst-case data loss is bounded to one day.
- **Weekly** — covers and `.env`. These change only when you catalog new titles or rotate keys; weekly is plenty.
- **Off-site copy** — at least monthly. Rsync the `backups/` directory to a remote NAS, a USB drive you swap in / out, or a cloud-storage bucket. A fire that consumes the home server should not also consume your backups.

Tip

A backup that has never been restored is not a backup. Once a quarter, restore your latest dump into a throwaway docker environment and verify the title and loan counts match what `/admin > Health` reports on production.

7.4 Restore procedure

Database

For the bundled-database setup:

```
docker compose stop mybibli
docker compose exec -T db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" \
    -e "DROP DATABASE mybibli; CREATE DATABASE mybibli CHARACTER SET utf8mb4;"
zcat backups/mybibli-20260513-1200.sql.gz | \
    docker compose exec -T db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" mybibli
docker compose start mybibli
```

For external MariaDB, the same `mariadb` client commands work against the external server.

Covers and `.env`

For a host-bind-mount install:

```
tar xzf backups/mybibli-files-20260513.tar.gz
```

For the default named-volume install, restore through a throwaway container in the symmetric shape to the backup:

```
docker compose stop mybibli
docker run --rm \
    -v mybibli_covers:/target \
    -v "$(pwd)/backups:/backup:ro" \
    alpine \
    sh -c "rm -rf /target/* && tar xzf /backup/mybibli-covers-20260513.tar.gz -C /target"
tar xzf backups/mybibli-env-20260513.tar.gz
docker compose start mybibli
```

Restart the stack so the mybibli container picks up the restored `.env`.

7.5 Migration to a new host

The backup procedure is also the migration procedure:

1. On the old host: produce a fresh database dump and tar archive.
2. Transfer both to the new host.
3. On the new host: install Docker, pull the compose file from the release matching the dump's version, place `.env` and `covers/` alongside, restore the database, start the stack.
4. Verify by counting titles on `/admin > Health` and spot-checking a few title pages.

Warning

The dump is tied to the *schema* version. If the source host ran 1.1.0 and the new host pulls 1.5.0, the new host's migrations will upgrade the restored database in place — that is fine. The reverse (restoring a 1.5.0 dump onto a 1.1.0 install) is *not* supported and will fail at boot. Always restore on a host running the same or newer mybibli release.

Chapter 8

Upgrade and migration

Routine upgrades are designed to be safe. The exception is the 1.0.x to 1.1.0 jump, which is a one-time breaking change documented below.

8.1 Routine upgrades (1.y to 1.z)

1. Read the release notes for every version between your current and the target version.
2. Take a fresh **database backup** (chapter 7).
3. Update the image tag in `docker-compose.yml` (or repull `:latest` if you track that tag).
4. Run `docker compose up -d`. The new container boots, runs any new migrations against the existing database, and starts serving requests.
5. Watch the logs for migration errors: `docker compose logs mybibli | grep -i migrate`.

Migrations are append-only — schema changes are forward compatible. A 1.4 install upgraded to 1.5 keeps every row from 1.4 and applies new tables / columns in place.

Tip

For homelab installs, the `:latest` Docker tag is a reasonable default. For production-style setups (a community library, an association), pin to an explicit version tag so an unattended pull does not bring in a release you have not read about.

8.2 Skipping intermediate versions

Going from a much older install (say, 1.3.x) directly to the current release is supported and safe for the database. The migration runner applies every pending migration in timestamp order at boot, regardless of how many you skip. Schema migrations are purely additive — no `DROP COLUMN`, no `DROP TABLE` in any release — and the four data backfills that exist (e.g. the 1.1.0 CSRF token seed, the 1.1.8 `manually_edited_fields` cleanup) are idempotent.

What still applies on a multi-version jump:

- **Back up before upgrading.** There is no automatic rollback. A migration that fails mid-execution leaves the database in an inconsistent state; the recovery path is the pre-upgrade backup (chapter 7).

- **Pre-1.1.4 cover JPGs are gone.** Until 1.1.4 the `mybibli_covers` volume was not declared, so cover images written before that version were lost on every container upgrade. Skipping straight to 1.7 does not recover them — re-fetch via the admin **Bulk re-fetch missing covers** action (added in 1.2) or the per-title **Re-fetch metadata** button.
- **The first boot may take a few minutes.** A jump that runs 20+ migrations against a large catalog is silent at the UX level — wait for `/health` to respond before worrying.

The 1.0.x to 1.1.0 jump is the one exception that needs the dedicated procedure below.

8.3 The 1.0.x to 1.1.0 jump

Warning

This is a one-time breaking change. 1.0.x installs are subject to the issue documented in #173: the seed migrations create the credentials `admin/admin` and `librarian/librarian` on every fresh install, including production, and silently bypass the first-launch setup wizard.

The recommended path depends on whether your 1.0.x install holds real data:

If your 1.0.x install is empty (no titles cataloged)

The simplest path: wipe the database and reinstall on 1.1.0+.

1. Stop the stack: `docker compose down`.
2. Remove the database volume: `docker volume rm mybibli_mybibli_db_data` (or the equivalent volume name on your host — `docker volume ls` lists them).
3. Pull the 1.1.0 image: `docker compose pull`.
4. Update the image tag in `docker-compose.yml` if you pinned a previous version.
5. Start: `docker compose up -d`.
6. Open `/setup` and walk through the wizard.

If your 1.0.x install holds real data

1. Take a database backup (chapter 7).
2. Take a covers + `.env` backup.
3. Log in as the seeded admin and rotate the password *before* you upgrade. The 1.1.0 upgrade flow itself is safe, but rotation closes the only window where the default credentials could leak.
4. Update the image tag in `docker-compose.yml` and run `docker compose up -d`.
5. The 1.1.0 boot detects the existing admin and does *not* re-enable the wizard — the rotated admin remains the active admin.

The new `MYBIBLI_SEED_DEV_USERS` variable

Starting with 1.1.0, the seed migration is gated by `MYBIBLI_SEED_DEV_USERS`. The default is `0` (no seed). Only set it to `1` for local-development workflows or automated tests — never in production. The first-launch wizard becomes the canonical onboarding path.

8.4 Reading the release notes

Every mybibli release publishes a GitHub Release page with:

- a one-paragraph summary of the change set;
- the migration impact (none / additive / breaking);
- a link to the diff against the previous tag;
- this manual as PDF, English and French, attached to the release itself — the copy that matches that exact version.

Read the notes before upgrading. Breaking changes are tagged with a prominent *Breaking* label.

8.5 What's new in 1.21.1

Version 1.21.1 is a patch release on top of 1.21.0. *No migration*, no setting changed — pull the image, recreate the container, done. It carries one fix, found by the first agent run against a real catalog.

- **#558 — a truncated JPEG cover is refused.** The image decoder accepted a JPEG cut short: it kept the rows it read and painted the rest grey. A cover proposed by an agent, downloaded from a provider or uploaded by hand could thus be a strip of image over a flat grey area. mybibli now checks that a JPEG runs to its end-of-image marker and refuses it otherwise (*Image data is incomplete* for an agent). PNG, WebP and GIF were already refused when cut short.

  Covers proposed before the upgrade are not re-checked: reject the ones that show half grey on the Proposals page before having them proposed again.

8.6 What's new in 1.21.0

Version 1.21.0 is a minor release on 1.20.1 and closes the epic that 1.20.1 started (#493). It brings *six additive migrations*, run automatically at the first boot (new tables for proposals and expected tomes, a new key scope): take a database backup first, as for every upgrade (chapter 7). There is no setting to change for an install that does not use an AI agent.

- **AI agents, under supervision.** mybibli now includes an MCP server at `/api/mcp`. An AI agent — Claude Code, Hermes Agent or any MCP client over streamable HTTP — connects with a new **Read + propose** API key (Administration → API keys), reads the catalog through twelve tools and *proposes*: a new series and the tomes you are missing, additions to a series, a missing subtitle, description, publisher, year, Dewey code or contributor, a cover — always citing its sources. It never changes the catalog itself. Chapter 13 is new and covers the whole workflow.
- **The Proposals page.** Librarians and Admins get a **Proposals** entry in the navigation bar, with a badge counting the pending proposals, and a **Pending proposals** indicator

on the home page. Each proposal shows its sources, the current and the proposed values side by side (with *New* / *Changed* badges) or the current and the proposed cover. You can edit or remove lines, accept, or reject with a reason the agent can read. A proposal whose subject changed in the meantime becomes *Outdated* and nothing is applied. Accepted values count as edited by hand: an automatic metadata download never overwrites them (section 13.7).

- **Resubmission delay.** After a rejection, an equivalent proposal is refused for 90 days, whichever key files it. The delay is a new setting under Administration → System → Proposals, from 0 to 3650 days (section 13.7.8).
- **Expected tomes.** A series can now name the tomes you do not own yet — by hand, with or without an agent, or from an accepted proposal. They fill their gap on the series page, go to the wish list in one click, and take their position automatically when you scan the book (section 13.8).
- **#496 — the navigation bar stays at the top** while you scroll. It now switches to the hamburger menu below 1280 pixels instead of 1024, so that every entry — including the new Proposals one — fits on one line in every language (section 3.1).

For the operator

- **The endpoint answers on the LAN only, and only to known names.** By default `/api/mcp` accepts requests addressed to `localhost` only. To let an agent on another computer connect, set `MYBIBLI_MCP_ALLOWED_HOSTS` to the name or address it uses to reach the NAS, then restart (section 13.4). The `docker-compose.yml` of 1.21.0 passes this variable and `MYBIBLI_MCP_ALLOWED_ORIGINS` through; if you keep an older copy, download the new one or add the two lines.
- **Proposed covers wait in the covers volume**, in a hidden `.staging` folder that is never served and is cleaned up by the scheduled purge. Nothing to configure; your backups of the covers volume include it (section 13.9).
- **One new dependency**, the MCP library `rmcp`, pinned to an exact version and upgraded only for a security advisory.

ISBN-10 at scan, the Library of Congress through SRU and the *Recent additions* triangle, also part of this epic, already shipped in 1.20.1 (below).

8.7 What's new in 1.20.1

Version 1.20.1 is a patch release on 1.20.0. *No migration*, no setting change — pull the image, restart, done. It carries three fixes reported from production.

- **#497 — an ISBN-10 is an ISBN.** Older books and comics often carry only the ten-character ISBN. Scanned on `/catalog` or on the home page, it used to be taken for a UPC code: a title with no metadata was created under the raw number. It is now recognised, including a final `X`, and converted to its ISBN-13 — `2205001213` becomes `9782205001211` — exactly as if the ISBN-13 had been scanned: metadata lookup, duplicate check, wishlist link. A home-page search by ISBN-10 finds the title too.

  Titles catalogued earlier under a raw ISBN-10 are *not* merged automatically. Look for them (a ten-character code, no metadata) and clean them up by hand. Until you do, rescanning such a book creates a second title under the ISBN-13.

- **#495 — the Library of Congress answers again.** Since 26 October 2026 the Library of Congress has put its JSON search behind a bot challenge, and the provider returned nothing. It now reads the same records through the Library's SRU service, in MARC 21 — the service that already supplied the cataloguing zones since 1.14.0. Title, authors, publisher, date, language, page count, description and Dewey come from it; the cover, which MARC does not carry, comes from the usual cover fallbacks.

  Two things to know. The SRU service is reached over *plain HTTP on port 210* (`lx2.loc.gov`): a network that only lets ports 80 and 443 out will show the Library of Congress as unreachable on the Health tab, and the chain simply moves on to the next provider. And English-language titles scanned between 26 October and this upgrade got no Library of Congress data: open such a title and press *Re-download metadata*, or run *Backfill metadata from libraries* from the Health tab.

- **#494 — the *Recent additions* triangle turns.** The triangle in front of the home page's *Recent additions* section now points down while the section is open, including when the page reopens it from your last visit.

- **Dependency fix.** The TLS library behind the outbound metadata lookups moves to `rustls` 0.23.46 and `rustls-webpki` 0.103.16 for advisories RUSTSEC-2026-0346 to 0349 and RUSTSEC-2026-0382. Nothing to configure.

8.8 What's new in 1.20.0

Version 1.20.0 is a minor release on 1.19.0. *No migration* — pull the image, restart, done. It carries one change request.

- **#489 — where the labels end.** The `/admin > Health` tab gains a *Barcode labels* section under the entity counters: the last volume number (V-code) and the last shelf number (L-code) in use, each with the next free number — `V0142 (next: V0143)`. Read it before printing a new sheet of pre-generated labels, so the sheet starts right after the occupied range.

  "Last" means the highest number ever printed, trashed items included: a sticker already stuck on a book or a shelf is never reissued. This is the rule the `/catalog` info line has followed since 1.12.0, so the two surfaces always agree. An empty catalog reads "None yet"; `V9999` reads "none left" rather than proposing a number the scanner would reject.

- **Dependency fix.** The TLS library behind the outbound metadata lookups (`rustls`) moves from 0.23.40 to 0.23.45 for advisory RUSTSEC-2026-0285. No code change, nothing to configure.

8.9 What's new in 1.19.0

Version 1.19.0 is a minor release on 1.18.0. *No migration* — pull the image, restart, done. It carries the three change requests of a security review conducted on the 1.18.0 code, plus the documentation work that review made necessary.

- **#478 — the Trash can actually restore now.** The Trash panel had shown a *Restore* button since 1.2.0. It pointed at an address the application never answered on, so clicking it did nothing at all: no restore, no error message, nothing. If you ever deleted a title by accident, opened the Trash and wondered why the button seemed dead — it was.

It works. Deleted something by mistake, restore it, and it returns where it was. When the record's relationships changed while it sat in the trash — a title reassigned to another series, say — mybibli asks before clearing the stale link rather than deciding for you.

- **#480 — the development accounts leave no trace.** A fresh install has always shipped two development accounts, `admin/admin` and `librarian/librarian`, which the application removes at first boot so the setup wizard can run (section 8.3). Removed, until now, meant *moved to the Trash*: the rows sat there for thirty days with their passwords intact — passwords published in this project's own security policy. With #478 making the Restore button work, an administrator could have restored them in good faith and handed the house keys back.

  They are now deleted outright, along with the pre-set development session. An instance upgrading from 1.18.0 or earlier clears whatever its old boot left behind, on the first start after the upgrade. Nothing changes for an operator who rotated the seeded password: that account is left alone, as it always was.

- **#479 — a cover image cannot exhaust the server.** Cover files were already capped at 10 MiB. That caps the *file*, which says nothing about memory: a few hundred bytes of image can legitimately describe a picture of several gigabytes, and decoding it would have taken the container down — along with every request in flight. The decoder now works under a fixed budget and refuses anything beyond it with a message saying so. A photograph from any phone still passes.

  This matters most where the images are not yours: covers downloaded from the libraries and catalogues in the metadata chain.

Note

Nothing to do on upgrade. No schema change, no configuration change, no manual step. Take your usual backup out of habit rather than necessity.

This release also brings the documentation in line with the software: K10plus in the provider chapter, the shelf-audit workflow described as it actually works, reference sections for management labels and for valuation, and a route reference that covers every route rather than half of them.

8.10 What's new in 1.18.0

Version 1.18.0 is a minor release on 1.17.0. *One automatic, additive migration* — three new tables, created at first boot. Nothing to do by hand, but see the note at the end of this section before downgrading.

- **#443 — labels for the work in progress.** A genre says what a book *is*. A condition state says how a copy has *aged*. Neither of them says *"I am not sure this record is right, come back to it"*, and until now there was nowhere to write that down.

  Define your own vocabulary once, in *Administration → Reference data* — "To check", "Re-read", "Damaged binding", whatever your practice calls for. Then put any of it on a title or on a single copy, from the same list, using the new section on their pages. An item can carry several.

  The *Labels* entry in the menu opens a page listing your whole vocabulary with how much each label is in use. Click one and you get everything carrying it: the titles first, the copies

below, each in its own table because they are not described by the same columns. Click any entry to reach the item itself and fix what needed fixing.

Tip

Labels are yours alone. A visitor browsing your catalogue sees none of them — not on a page, not in a count, not by guessing an address. The vocabulary is deliberately not translated: these are your words, not the software's.

A label you have used somewhere cannot be deleted; mybibli tells you how many titles and copies still carry it, so you can detach them first if that is really what you want.

- **The menu appears earlier on narrow screens.** With the new *Labels* entry, an administrator's navigation bar no longer fits a tablet held upright. The bar therefore folds into the "hamburger" menu below 1024 pixels instead of 768. On a tablet you will now find the menu button where the links used to be; nothing is lost, everything is one tap away.

Warning

First migration since 1.15.0. Downgrading to 1.17.0 leaves the three new tables in place, unread — harmless, and your labels are still there if you come back. But take the usual pre-upgrade backup (chapter 7) rather than relying on that: the safe path back is always the backup, not the assumption.

8.11 What's new in 1.17.0

Version 1.17.0 is a minor release on 1.16.0. *No migration, no operator action* — upgrade the image and restart.

Both changes came from the same reader, eight months apart, and neither turned out to be the fault it was reported as. In both cases the machinery was already doing the right thing and the screen was failing to say so.

- **#202 — a failed lookup now tells you why.** When no library could describe a scanned book, mybibli said only that it had failed. That single sentence covered three situations that call for opposite reactions, and you had no way to tell them apart:
  - every library was searched and none holds the book — nothing to be done, and no key or retry will change it. Type the details in by hand;
  - a library was too busy or too slow to answer — genuinely worth pressing the button again in a few minutes;
  - a library was never asked at all, because its access key is not filled in — one field in *Administration → System*, and it is settled for good.

  Open a title and press *Re-download metadata*: the message underneath now names which of the three it was, and names the libraries concerned. Their names are never translated — they are the libraries' own.

  One case had to be repaired for this to be honest at all. OMDb and TMDb, with no access key entered, used to report exactly what they report after a fruitless search. They now say that they were not asked.

- **#206 — which of genre and Dewey a lookup may overwrite.** The request was that the genre stop being overwritten by automatic metadata. It never was: no library in the chain supplies a genre, and nothing in mybibli writes one. What the form did not say is that the field beside it behaves the *other* way round — the Dewey code *is* offered by libraries, and goes on being offered until the day you correct it yourself, after which it is left alone as yours.

  Reading the form, one would naturally guess this the wrong way round. Both fields are now grouped in a *Classification* section that states the difference in one line. The publication date, which describes the book rather than your shelving of it, stays outside.

8.12 What's new in 1.16.0

Version 1.16.0 is a minor release on 1.15.0. *No migration, no operator action* — upgrade the image and restart.

- **#450 — a third library describing your English-language books.** Some titles stayed bare however often you asked: neither the French National Library nor the Library of Congress holds them. mybibli now also consults *K10plus*, the shared catalogue of the German academic libraries, purely to fill in the bibliographic fields the first answering library left empty. On the collection it was measured against, it filled more of them than any other source, including for English titles published in Britain and the United States — German university libraries hold a great deal of anglophone technical publishing. **To benefit from it on titles already catalogued**, run *Backfill metadata* once from *Administration → Health* after upgrading; new scans use it automatically.

  A caveat worth stating plainly: a residue of titles remains that no consulted library holds — small-press and self-published works, chiefly British. Six catalogues have now been tried against them. For those, the fields must be filled by hand; no further update will change that.

- **#459 — getting back in after losing the administrator password.** Until now, an administrator locked out of their own installation had to open a database console — an intimidating detour on a home NAS, for a situation that is merely annoying. A single variable, `MYBIBLI_RESET_ADMIN`, set in the `docker-compose` file, resets one account at the next restart. It works once: leaving the variable in place does not re-arm it, and it therefore cannot become a permanent open door. The *Operations & debugging* chapter gives the exact syntax.

- **#457 — a shelf-location code that is actually free.** When creating a location, mybibli proposes the next available code. It was overlooking deleted locations, which keep their code until the definitive purge — so the proposed code was sometimes already taken and saving failed on an error that explained nothing. The proposal now counts them and offers the lowest genuinely free code.

- **#458 — a more trustworthy test suite.** Four automated tests failed at random, without any actual defect. Nothing changes in the application; the point is that a failure now means something, instead of prompting a re-run.

8.13 What's new in 1.15.0

Version 1.15.0 is a minor release on 1.14.1, drawn from a review of a month of production logs. *One automatic, additive migration; no operator action.* The new column is nullable and unread

by the previous binary, so rolling back to 1.14.1 stays clean.

- **#202 — a title now says where its description came from.** The catalog consults several libraries in turn and keeps the first useful answer, but until now it forgot which one had replied. Facing a sparse record you could not tell a library that answered briefly from one that never answered at all. The title page now names the source. Titles catalogued before this release show nothing, which is honest: their provenance was never recorded, and inventing one would be worse than leaving it blank. The library's name is never translated — it is a proper noun.
- **#424 — readability of faint text.** Several secondary texts — the counter beside *Recent additions*, the placeholders in the volume table, the controls in the locations tree — were too pale against their background to meet the AA accessibility threshold. They have been darkened. The effect is subtle by design: these texts must stay discreet, only readable.
- **#419 — fewer titles abandoned during a bulk metadata run.** When a provider throttles a burst of requests, the run retries the affected titles. A third, longer attempt has been added: on the measured run, nine titles were being written off for want of it. Only throttled titles wait, so a clean run is unaffected. Measurement also settled a plausible but wrong idea: spacing the requests further out does not help, because the refusals do not follow the request rate.
- **#449 — a shorter commit in the log.** The startup line printed the full forty-character commit; it now prints the seven characters everyone actually compares.

8.14 What's new in 1.14.1

Version 1.14.1 is a small patch on the 1.14.0 minor. *No migration, no operator action.*

- **#447 — the application identifies itself in its log.** At startup mybibli now records its version, the commit the image was built from, and the build profile. A log file read in isolation — weeks later, or pasted into an incident report — is therefore unambiguous about which build produced it. Establishing that previously meant opening the host's `docker-compose` file, a detour made worse by releases carrying no migration, which leave no other trace to infer the version from. The commit appears only on official images; a local build reports `unknown`, having no commit it could honestly claim.

8.15 What's new in 1.14.0

Version 1.14.0 is a feature minor on 1.13.0. It fixes three defects in the cataloging flow and extends bibliographic enrichment to English-language books. *No database migration — the upgrade is an image swap against an unchanged schema, and rolling back to 1.13.0 stays clean.* Routine `docker compose pull && docker compose up -d`.

Cataloging fixes

- **#440 — volume labels attached to the wrong title.** When several UPC-identified items (typically CDs) were cataloged in a row within one browser session, only the first established the scan context correctly. The rest left the previous title active, so the label scanned next attached to that title, or failed with a misleading message if it had since been deleted. Every scan path now goes through one shared context activation.

- **#441 — misleading error message.** When the active title had been deleted, scanning a label answered "not found", which pointed at the label when the real problem was that no item was active. The message now says so, and the stale context is cleared so the next scan starts clean.
- **#442 — label locked after deletion.** Deleting a volume did not release its V-code: the label could only be re-used after an administrator permanently purged it from the Trash, and the error named its owner "`?`". The label is now directly reusable. The previous copy's details (condition, location, purchase price, valuation) are discarded — it is a different physical object — and recorded in the audit trail. Reuse is refused when the deleted volume carries loan history, since it would re-attribute that history to a different copy.

Bibliographic zones for English-language books

- **#439 — the Library of Congress complements the BnF.** The six UNIMARC-aligned fields introduced in 1.13.0 stayed empty on English-language books: the BnF does not catalog them, and the other sources carry no structured bibliographic data. mybibli now also reads **MARC 21** records from the Library of Congress.
- **Field-by-field complementarity.** When the source that resolved a title fills only some of the six zones, the other library is consulted to complete the ones left empty. A value already present is never overwritten.
- **Actual reach.** Only books with ISBN prefixes 9780 and 9781 are affected. *French titles the BnF could not describe will stay empty* — the Library of Congress does not hold them either. Bear that in mind when reading the backfill figures after upgrading.
- **Renamed button.** The Health-tab action is now *Backfill metadata from libraries*, since it no longer relies on the BnF alone.

8.16 What's new in 1.13.0

Version 1.13.0 is a feature minor on 1.12.0 centered on one theme: aligning the catalog with the **UNIMARC** bibliographic standard. *One automatic, additive database migration — no operator action required.* Routine `docker compose pull && docker compose up -d`.

- **#389 — UNIMARC-aligned cataloging fields.** Titles now carry six additional bibliographic fields mapped to their UNIMARC zones: statement of responsibility (200$f), edition statement (205$a), collection title and number (225$a/$v), general note (300$a) and original title (500$a). They are captured automatically from the BnF when a book is scanned, and shown on the title page whenever present. The authoritative field-to-zone mapping ships in the repository as `docs/unimarc-mapping.md`; record import/export (ISO 2709 / UNIMARC XML) is a separate, later work package.
- **New Health-tab action: *Backfill metadata from BnF.*** Re-runs the metadata lookup for every already-cataloged title that has a lookup code, filling the new UNIMARC fields (and anything else the providers have learned since) without ever overwriting a value you edited by hand. Shares the pacing, throttle back-off and single-run lock of the cover refetch action.
- **#434 — cataloging log summary.** Every metadata resolution now logs a single `info` summary line (cover found? how many UNIMARC fields filled?), which makes a backfill run easy to audit from the log file.

8.17 What's new in 1.12.0

Version 1.12.0 is a 2-issue feature minor on 1.11.0. *No migration, no operator action.* Routine `docker compose pull && docker compose up -d`.

- **#427 — two new cover sources.** When metadata resolves without a cover, mybibli now also asks the *BnF Couvertures* service (the national library's legal-deposit cover scans — the strongest source for French and Swiss-French publishers) and *Inventaire.io* (community-driven, strong on French-language comics and manga) before the Open Library fallback. Live-tested against a real production catalog: roughly half of the previously unrecoverable covers become available. Run *Re-fetch missing covers* from the Health tab after upgrading to collect them. See chapter 5.
- **#428 — label-printing helper.** The catalog page now shows the highest V-code and L-code already in use (including trashed items — a printed sticker never gets reissued), so freshly printed label sheets can start right after them.
- Also fixed: covers served from hosts without TLS (e.g. a LAN-local image server) no longer fail to download.

8.18 What's new in 1.11.0

Version 1.11.0 bundles the four issues raised by the July production-log review — two bug fixes and two operator-facing improvements. *One additive settings migration; applies automatically — no operator action.* Routine `docker compose pull && docker compose up -d`.

- **#418 — sessions survive a tablet screen-lock.** The session cookie is now persistent (its lifetime follows the inactivity timeout) instead of a browser-session cookie, so iPads and other devices that discard session cookies on lock no longer log you out mid-cataloguing. The inactivity timeout itself is now admin-configurable in hours (1–720, default 4) under `/admin > System > Sessions` — see section 2.5.
- **#419 — bulk cover-refetch actually recovers covers.** The *Re-fetch missing covers* action paces its requests (configurable delay, default 1 s per title), retries throttled lookups with increasing back-off, and finishes with a summary on the Health tab — covers recovered, provider errors, and titles with no cover available anywhere. Previously a back-to-back run tripped Google Books' burst throttling and silently recovered nothing.
- **#416 — daily auto-purge unblocked.** Leftover session rows referencing a long-deactivated user no longer block the 30-day hard-purge of that user (a daily error in the logs since June). The purge now cleans those rows itself.
- **#417 — quieter logs.** Clicking the dashboard *Gaps* chips no longer emits a spurious warning line per click.

8.19 What's new in 1.10.0

Version 1.10.0 is a single-feature minor on 1.9.1. *No migration, no operator action.* Routine `docker compose pull && docker compose up -d`.

- **#9 — undo a scan action.** When you shelve a volume or activate a batch location on the catalog page, the confirmation message now shows an **Undo** button. Click it within 30 seconds to reverse that last action — the volume goes back to its previous location, or

the batch-location activation is undone. Only the most recent action can be undone, and undo never deletes a title or volume you just created. See subsection 3.9.1 in the usage chapter.

8.20 What's new in 1.9.1

Version 1.9.1 is a small patch on the 1.9.0 feature minor (saved searches, per-provider metadata timeouts, logging fixes — see the GitHub release notes for 1.9.0). *No migration, no operator action.* Routine `docker compose pull && docker compose up -d`.

- **#403 — German and Italian everywhere.** The last two interface messages that only existed in English and French — the "please close the current dialog" notice in the admin reference-data screens and the "server error, please try again" feedback — now follow the interface language like everything else, including German and Italian.
- **#412 — test-suite reliability.** An internal timing race in the automated test suite was fixed. No change in how the application behaves — listed for completeness.

8.21 Older release notes

Release notes for 1.9.0 and earlier are published on the project's GitHub Releases page (`https://github.com/guycorbaz/mybibli/releases`), one entry per tag. The upgrade procedure is identical for every 1.y release: routine `docker compose pull && docker compose up -d`; migrations apply automatically, and any release requiring operator action says so prominently in its notes.

8.22 Rollback

Rollback is *not* a first-class flow. Schema migrations move forward only — there is no `down.sql`. The supported recovery strategy is:

1. Restore the pre-upgrade database backup (chapter 7).
2. Pin the image tag back to the prior version.
3. Start the stack.

This works because the backup is consistent with the prior schema. Trying to point an older image at a newly-migrated database will fail to boot.

Chapter 9

Best practices

Concentrated lessons from real-world deployments. Every section is optional — pick what fits your scale.

9.1 Storage hierarchy

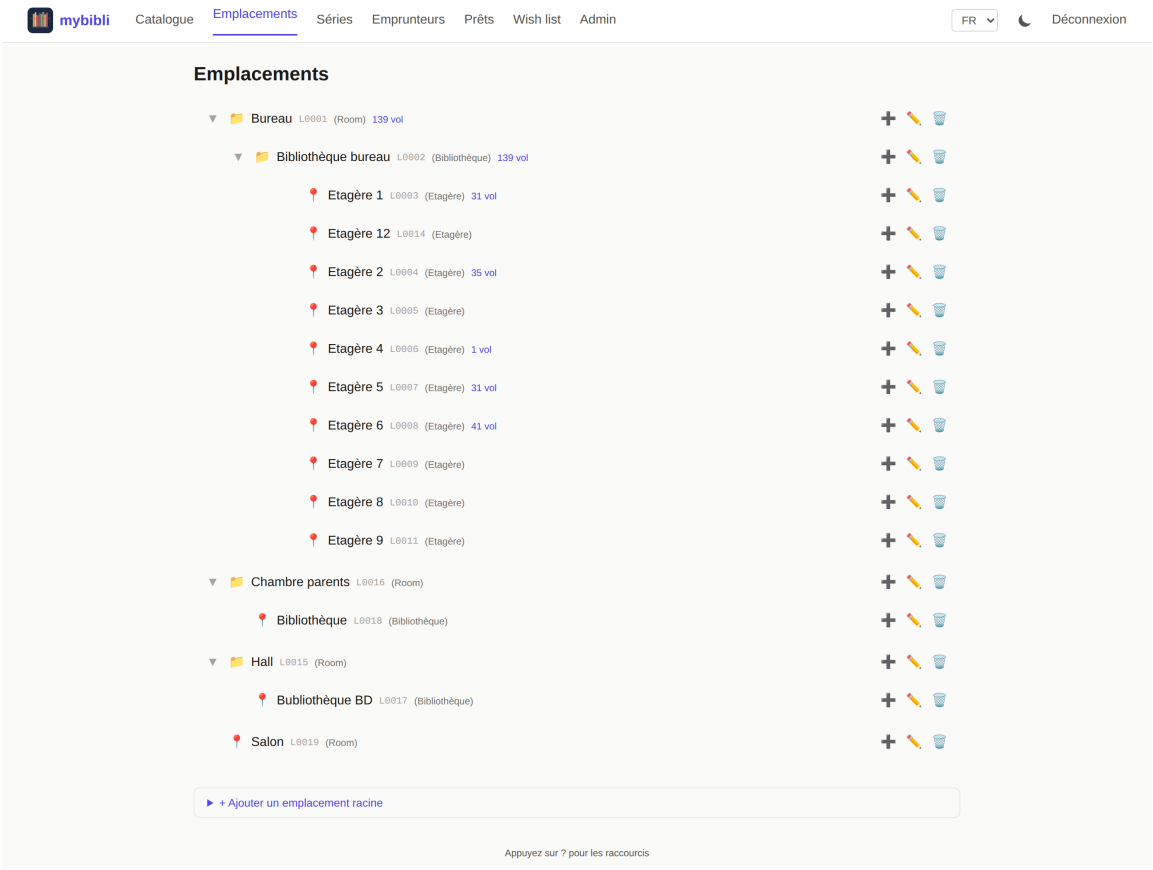


Figure 9.1: The Locations page on a live install: a household hierarchy with a study room, an in-room bookcase, ten shelves, and two flat-level rooms — each node carrying its L-code and a live volume count.

- **Three levels are usually enough.** Room → shelf → row covers most households. A fourth level (e.g. “box inside row”) becomes paperwork you have to maintain.

- **Name locations the way you talk about them.** "Living-room shelf, top" beats "LR-S01-R01". The names are visible on barcode labels; treat them as user-facing.
- **Print L-codes once.** Re-printing because you rebalanced shelves is wasted effort — L-codes are stable identifiers; the human-readable name on the label can drift and that is fine.

9.2 V-code rolls

- Print V-codes in rolls of 50 to 100. Pre-printed stickers in a holder beside the scanner station make cataloging *fast*.
- **Stick the V-code before scanning.** It removes a future failure mode where you mislabel a book and have to scan-find it again.
- **Place the sticker consistently.** Inside back cover, upper-right corner, parallel to the spine — pick a rule and stick to it. Future-you doing an audit will thank present-you.

9.3 Genre versus Dewey

mybibli supports both broad genres and Dewey codes. Pick one as your primary navigation aid:

- **Use genres** when the audience browses by mood ("I want a thriller") and the collection is small enough that "Thriller" is granular enough.
- **Use Dewey** when the audience browses by topic ("books on 18th-century French history") or when the collection has reached the size where "Non-fiction" is no longer a useful filter.
- Using both is fine; mybibli does not force a choice.

The two behave differently when metadata is fetched, and the difference is easy to guess backwards:

- **The genre is yours alone.** No library in the provider chain supplies one, and no metadata fetch — not the automatic one at scan time, not *Re-fetch metadata*, not the admin backfill — ever changes it. Assign it once and it stays.
- **The Dewey code may be proposed for you.** A library that holds the title can supply it, and it will fill an empty field. The moment you edit it yourself, mybibli records that the value is yours and stops overwriting it.

Both fields sit together in the *Classification* section of the metadata-edit form, apart from the bibliographic fields the libraries describe, and that section states the distinction on screen.

9.4 Audit cadence

- **Quarterly** for collections under a few hundred volumes.
- **Monthly** for collections above that, especially if more than one person catalogs.
- Always run an audit *after* a major shelf rebalance — moving books physically without telling mybibli is the single biggest source of "where is this volume?" bugs.

9.5 User accounts

- **One human, one account.** Sharing a single `admin` account between two people loses the audit trail and makes the last-active-admin guard less useful.
- **Most editors should be Librarians, not Admins.** The admin panel handles installation-time concerns and rare cleanups; you do not need admin rights to catalog or loan.
- **Rotate the seeded admin password on first login.** On 1.1.0+ the setup wizard forces this; on legacy 1.0.x installs do it manually before exposing the instance to a network.

9.6 Loans

- **Keep borrowers identifiable.** "Marie L." beats "Marie" the moment the second Marie shows up. Email or phone in the notes field helps for chase-ups.
- **Run the overdue list weekly.** The dashboard indicator nags you for a reason — overdue loans compounded over a year become books you no longer own.
- **Mark damage at return time.** Switching a returned volume to *Worn* or *Damaged* captures the information while the borrower is in front of you; chasing the cost later from memory rarely works.

9.7 Metadata providers

- **Keep keys minimal.** You do not need all three keyed providers from day one — BnF + Open Library cover books for free. Add OMDb + TMDb the day you catalog your first DVD.
- **Treat keys as secrets.** `.env` is in `.gitignore`; keep backups of `.env` encrypted (chapter 7).

9.8 Upgrades

- **Read release notes before pulling `:latest`.** See chapter 8.
- **Back up immediately before any upgrade.** Even "additive" migrations have non-zero failure surface; a same-minute backup turns a worry into a chore.
- **Pin a version tag for production.** Auto-updates for a household instance are fine; for a community library, an unattended major bump can interrupt service.

Chapter 10

Troubleshooting

When things go wrong, start with the logs and the `/admin > Health` dashboard. This chapter catalogues the problems users actually hit, in rough order of frequency.

10.1 Reading the logs

```
docker compose logs --since 10m mybibli
docker compose logs -f mybibli
```

Logs are emitted as structured JSON. The most useful fields are `level`, `target` (which module logged this), `message`, and any contextual key=value pairs that the log line carries (`user_id`, `volume_id`, …).

To filter:

```
docker compose logs --since 10m mybibli | grep -i error
```

10.2 App does not start

Symptom: container exits immediately

Run `docker compose logs mybibli` and look for one of:

- `missing required environment variable: DATABASE_URL` — the `.env` file is missing or the variable is unset. See section 1.5.
- `Failed to create database pool` — the database hostname / port / credentials in `DATABASE_URL` are wrong, or the DB is not reachable. From the host, try `docker compose exec mybibli sh -c 'apk add curl; curl <db-host>:<db-port>'` to confirm reachability.
- `Failed to run database migrations` — usually a permissions issue. Confirm the DB user has `ALL PRIVILEGES` on the database (see section 1.4).
- `Address already in use` — the published host port is taken. Change `HOST_PORT` in `.env`.

Symptom: every page returns 404

The container is up but routes are missing. Most common cause: you pulled an image tag that does not exist (typo) and Docker fell back to a stale older tag. `docker compose pull` and restart.

10.3 Cannot log in

Symptom: "Invalid credentials" on a fresh 1.0.x install

The seed credentials are `admin/admin`. If they do not work either:

- the database migration that creates them failed silently — look for migration errors in the logs;
- the previous admin already rotated the password (check with whoever installed mybibli);
- you are looking at a 1.1.0+ install, where the seed credentials are planted by the migration but immediately soft-deleted by the issue-#173 seed gate at startup — use the wizard at `/setup` instead.

10.4 Stray users in the `users` table after the wizard

Symptom: four users in the database, only one was created in the wizard

If you inspect the `users` table directly (via phpMyAdmin or the MariaDB CLI) right after completing `/setup`, you will see *four* rows even though you only entered one username and password. This is by design. Three of the rows cannot be used to log in.

username	role	active	deleted_at
`admin`	`admin`	1	set, ≈ first-boot time
`librarian`	`librarian`	1	set, ≈ first-boot time
`SYSTEM`	`system`	0	NULL
`<your-username>`	`admin`	1	NULL

- `admin` and `librarian` are planted by the seed migrations (which run on every fresh install for the dev / E2E workflow) and immediately *soft-deleted* by the issue-#173 seed gate at startup. Their `deleted_at` is set, so the login query (which filters `deleted_at IS NULL`) refuses them.
- `SYSTEM` is a non-loginable utility account used to attribute the audit-log entries produced by the 30-day auto-purge background task. Its role is `system` (outside the login enum `admin/librarian`), its `password_hash` is the literal string `NO_LOGIN_SYSTEM_USER_HASH_NOT_A_VALID_ARGON2_` and its `active` flag is `0`.

Tip

To confirm the soft-deleted rows are inert, try logging in as `admin/admin` — it returns an authentication failure. The `/admin > Users` panel filters out both the `SYSTEM` row and any `deleted_at`-set row, so the admin UI shows the single account you created in the wizard.

The two soft-deleted rows are physically removed from the table by the 30-day auto-purge scheduler — expect them to disappear roughly one month after the boot date stamped in `deleted_at`.

Symptom: login redirects back to `/login`

The session cookie is being rejected by the browser. Causes:

- `MYBIBLI_COOKIE_SECURE=true` on a plain-HTTP deployment — the browser refuses to accept a Secure cookie over HTTP. Set the variable to `false` or front the instance with HTTPS.
- Cookies blocked by browser policy (private window with strict third-party blocking, etc.). Test in a default profile.

Symptom: locked out — last admin deactivated

The last-active-admin guard should prevent this, but if a previous admin was deactivated then permanently deleted, recovery requires direct SQL access:

```
docker compose exec db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" mybibli \
    -e "UPDATE users SET deleted_at=NULL, active=1 WHERE username='your-admin';"
```

Then log in normally.

10.5 Metadata never resolves

Symptom: skeleton title stays unresolved

Open `/admin > Health`. Look at each provider's status. A red cross usually means one of:

- the provider is genuinely down (rare for BnF / Open Library; more common for the smaller services);
- the API key is missing or invalid — open `/admin > System > Metadata Providers` and re-enter the key;
- the host cannot reach the upstream. Confirm with `docker compose exec mybibli sh -c "wget -q https://www.googleapis.com/books/v1 -O -"` or equivalent.

Symptom: only French ISBNs resolve

You disabled (or never set) the Google Books / Open Library keys. BnF is excellent for French-published titles but limited for foreign-language titles. Add at least Open Library (no key required).

10.6 Search returns too little

Symptom: known titles do not surface

- Confirm the title is not soft-deleted (check `/admin > Trash`). Search hides soft-deleted items.
- Confirm the title's contributors are spelled the way you searched. mybibli's search is fuzzy but not magical; "Camus" will find "Albert Camus" but "Camu" might not.

Symptom: search is slow

For collections above 10,000 titles, queries that combine contributor + free-text can take a second or two. This is a known limitation; planned indexes will close the gap.

10.7 Cover images are missing

Symptom: every title shows the placeholder

The cover directory is empty or unreadable. Confirm:

- `docker-compose.yml` declares the `mybibli_covers` named volume (or a bind mount on `COVERS_HOST_PATH`) mapped to `/app/covers`. Pre-1.1.4 installs did NOT declare this volume — see the next subsection;
- the metadata fetch actually returned a cover URL (check the title page — if the URL is missing, the provider did not supply one).

Symptom: covers vanished after upgrading the image

You upgraded from a pre-1.1.4 install and all previously cataloged covers are gone. This is issue #213 — the older compose template did not persist `/app/covers`, so the writable layer holding those JPGs was destroyed when the new container replaced the old one.

Fix forward: pull the 1.1.4 `docker-compose.yml`, run `docker compose down && docker compose up -d`. Subsequent covers are persisted in the `mybibli_covers` volume.

Recovery of lost covers: open each affected title's detail page and click *Re-fetch metadata* to repopulate from the provider chain. A bulk action is tracked at issue #214.

10.8 Where to file bugs

Issues you cannot resolve, or behaviour that contradicts this manual: open a ticket at https://github.com/guycorbaz/mybibli/issues. The issue template asks for the version (visible in the page footer), the install method, and the steps to reproduce — please fill them in.

Chapter 11

API & integrations

mybibli ships a small JSON HTTP API at `/api/v1/*` that lets external scripts and AI tools read your catalog and update the classification of titles. The API runs on the same web server as the human-facing UI — there is no separate daemon, no extra port to open.

This chapter walks through:

- creating an API key in the admin panel,
- the scopes (*read only*, *read + write*, and *read + propose* for AI agents),
- the endpoints exposed under `/api/v1/*`,
- three end-to-end examples (`curl`, Python, an AI classifier loop).

11.1 Why an HTTP API?

The everyday use case is offloading classification work to an external tool — for instance, asking an LLM to suggest a Dewey code, a genre, or a clean subtitle for every newly-added English book. The LLM walks the catalog over `GET /api/v1/titles`, fetches the detail of each via `GET /api/v1/titles/{id}`, computes a suggestion, and writes it back with `PATCH /api/v1/titles/{id}`. Every PATCH is audited so you can roll back changes if a model produced garbage.

The API is *not* meant for high-throughput batch ingest — for that, use the SQL database directly during a maintenance window. It *is* meant for the kind of slow, careful, opinionated write that fits inside an LLM agent loop.

11.2 Creating an API key

API keys are managed under **Administration** → **API keys**. Only Admin users see this tab.

1. Click **Administration** in the navigation bar.
2. Open the **API keys** tab.
3. Fill in a **label** — a name only you see, e.g. *"AI classifier"* or *"Backup script"*. The label is for the audit log; the key plaintext itself is random and unrelated to it.
4. Choose a **scope**:

- **Read only** — `GET` endpoints only. Safe for read-only dashboards, mirroring tools, or curiosity-driven scripts.
- **Read + write** — additionally allows `PATCH /api/v1/titles/{id}` for a narrow allow-list of fields (genre, dewey, description, subtitle). Use this for LLM agents or curation tools.
- **Read + propose** — for an AI agent connected through the MCP endpoint `/api/mcp`: it reads like a read-only key and files proposals a librarian reviews, never changing the catalog. See chapter 13; on `/api/v1/*` it behaves as a read-only key.

5. Click **Create key**. A modal appears showing the full plaintext exactly once — copy it now. Once you close the modal there is no recovery path; if you lose the key, revoke it and create a new one.

The plaintext shape is `mybibli_<scope>_<random40>`, where `<scope>` is `ro`, `rw` or `pr`. Only the argon2 hash and the first 12 characters of the plaintext (the prefix) are persisted; the prefix is what mybibli uses to narrow candidates during authentication. The hash itself cannot be reversed.

11.3 Authenticating

Every `/api/v1/*` request must carry one of:

- `Authorization: Bearer <plaintext>` — preferred.
- `X-API-Key: <plaintext>` — convenience fallback.

A missing or invalid key returns `401 Unauthorized` with a JSON envelope:

```
{ "error": "unauthorized",
  "reason": "missing_api_key" }
```

A read-only key on a write endpoint returns `403 Forbidden` with the missing scope named:

```
{ "error": "forbidden",
  "reason": "insufficient_scope",
  "scope_required": "write" }
```

CSRF protection is intentionally bypassed for `/api/*` because bearer-token authentication does not ride on cookies and the cross-site replay risk that CSRF defends against does not apply.

11.4 Endpoints

11.4.1 GET /api/v1/titles

Paginated list. Supports `?page=N`, `?q=<search>`, `?genre_id=N`, and `?dewey_prefix=8`.

Returns:

```
{ "items": [ { "id": 42, "title": "...",
               "subtitle": null,
               "language": "en",
               "media_type": "book",
               "isbn": "9780...",
               "publisher": "...",
```

```
              "publication_year": 2020,
              "genre_id": 7,
              "genre_name": "Fiction",
              "dewey_code": "813.54" },
            ... ],
  "page": 1, "total_pages": 12,
  "total_items": 287 }
```

11.4.2 GET /api/v1/titles/{id}

Full detail, including contributors, volumes (with their location), and series assignments. The `version` field is the optimistic-locking handle — keep it next to the data your script will edit.

11.4.3 GET /api/v1/genres, /locations, /series

Reference data. Use these when an LLM needs to know which genres exist before it suggests a `genre_id`.

11.4.4 GET /api/v1/dewey/{prefix}

Bucket lookup. Pass a partial Dewey prefix (e.g. `8` for literature) and get back every title whose `dewey_code` starts with it. Useful for "find all books that already have a Dewey code in the 800s so I can compare".

11.4.5 PATCH /api/v1/titles/{id}

Write-scope only. Updates the four whitelisted fields:

- `subtitle` — nullable.
- `description` — nullable.
- `dewey_code` — nullable; digits, dots, slashes only.
- `genre_id` — must reference an existing, non-deleted genre.

The JSON body uses partial-update semantics:

- Field **omitted** → no change.
- Field set to `null` → clear to NULL (for the three nullable fields above; `null` on `genre_id` returns `400`).
- Field set to a value → update.

`version` is required. On mismatch the response is `409 Conflict` carrying both the expected and the supplied version — refetch and retry.

A successful PATCH writes one row into `admin_audit` with action `api_patch_title`, attribution via the issuing admin (`user_id`) and the API key (`details.via_api_key_id` + `details.via_api_key_label`), and a per-field `old`/`new` diff. A no-op PATCH (no field actually changed) returns `200` but does NOT bump `version` and does NOT write an audit row.

11.5 Examples

11.5.1 curl

```
KEY="mybibli_ro_..."

# List the first page of titles.
curl -H "Authorization: Bearer $KEY" \
     https://library.example.org/api/v1/titles

# Show one title in full.
curl -H "Authorization: Bearer $KEY" \
     https://library.example.org/api/v1/titles/42

# PATCH a Dewey code (needs a write-scope key).
KEY_WRITE="mybibli_rw_..."
curl -X PATCH \
     -H "Authorization: Bearer $KEY_WRITE" \
     -H "Content-Type: application/json" \
     -d '{"version": 3, "dewey_code": "813.54"}' \
     https://library.example.org/api/v1/titles/42
```

11.5.2 Python

```
import os, requests

BASE = "https://library.example.org/api/v1"
KEY  = os.environ["MYBIBLI_API_KEY"]
H    = {"Authorization": f"Bearer {KEY}"}

# Walk every page.
page = 1
while True:
    r = requests.get(f"{BASE}/titles?page={page}",
                     headers=H, timeout=30)
    r.raise_for_status()
    data = r.json()
    for t in data["items"]:
        print(t["id"], t["title"], t.get("dewey_code"))
    if page >= data["total_pages"]:
        break
    page += 1
```

11.5.3 An LLM classifier loop

The intended write-flow looks like this — pseudocode:

```
for t in walk_titles(KEY_READ):
    if t["dewey_code"]:
        continue                    # already classified
    detail = get(f"/titles/{t['id']}")
    suggestion = ask_llm(detail)   # returns {"dewey_code": ...}
    if not suggestion.get("dewey_code"):
```

```
        continue
    patch(
        f"/titles/{t['id']}",
        json = {
            "version":    detail["version"],
            "dewey_code": suggestion["dewey_code"],
        },
        key  = KEY_WRITE,
    )
```

If two classifiers race on the same title, the loser gets a `409` and re-fetches; no silent clobber. The audit log records who (key) suggested what — handy for rolling back a model that has a bad day.

11.6 Revoking a key

Revocation is one click in the admin panel. A revoked key *cannot* be restored; the audit history of past usage is preserved alongside the now-defunct row. If you suspect a key has leaked (committed to a public repo, e-mailed by mistake, found in a log file), revoke immediately and create a new one.

11.7 What the API does *not* expose

By design, the v1 API is read-mostly. It does not:

- create or delete titles, volumes, contributors, borrowers, loans, or series;
- update any field outside the four-field PATCH allow-list;
- return passwords, password hashes, session tokens, or any other credentials;
- accept multipart uploads (no cover-image PUT).

These are intentional v1 limits — the use case that justified the API is classification, and that fits inside the narrow allow-list. Wider write-surface would change the security posture (CSRF considerations, mass-mutation rate limits) and is deferred until a concrete need surfaces.

Chapter 12

Operations & debugging

This chapter documents where logs live on disk, how to control their verbosity, and the commands you typically need when a user reports odd behavior and you want to see what the app actually did.

12.1 Where the logs live

Starting with **v1.7.0**, mybibli writes its logs to two places simultaneously:

- **stdout** (unchanged) — visible via `docker compose logs mybibli`. Best for live-tailing during an active session.
- **A daily-rotating file on disk**, by default under `/var/log/mybibli/` inside the container, mounted on a named volume (`mybibli_logs`) on the host so files survive `docker compose pull && up -d`. Best for *post-mortem* debugging.

Files are named `mybibli.log.YYYY-MM-DD` (one per UTC day). A background task purges files older than 30 days automatically.

Changing the in-container log path

The default in-container log directory `/var/log/mybibli/` is controlled by the `MYBIBLI_LOG_DIR` environment variable. Change it only if you remount the volume elsewhere — the shipped `docker-compose.yml` maps the `mybibli_logs` named volume (or a bind mount via `LOGS_HOST_PATH`) onto `/var/log/mybibli`, so changing one without the other lands logs in the writable layer where they get wiped on every `docker compose up -d` after a `pull`.

See section 1.3 for the bind-mount setup (host-visible logs for DSM Log Center / journald shipping).

12.2 Tailing logs from the host

```
# Tail the current day's file in real time
docker compose exec mybibli tail -f /var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d)

# Last 100 lines of the current day's file
docker compose exec mybibli tail -n 100 /var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d)

# Show all available days
docker compose exec mybibli ls /var/log/mybibli/
```

```
# Search across all retained days for a keyword
docker compose exec mybibli sh -c "grep ERROR /var/log/mybibli/mybibli.log.*"
```

If you mounted the volume as a bind mount (`LOGS_HOST_PATH` in `.env`, see section 1.3), the files are visible directly on the host without needing `docker compose exec`.

12.3 Log levels

Two environment variables control verbosity:

- `MYBIBLI_LOG_LEVEL` (preferred, v1.7.0+) — accepts either a plain level (`trace`, `debug`, `info`, `warn`, `error`) or a `tracing-subscriber` `EnvFilter` directive list, e.g. `mybibli=debug,sqlx::query=warn`.
- `RUST_LOG` — legacy fallback, same format. Honored when `MYBIBLI_LOG_LEVEL` is unset.

Default is `info` — production-safe (info + warn + error, no per-query SQL noise, no per-request trace floods).

Recommended levels

`info` Production default. Catches authentication events, metadata fetches, scheduled-task runs, user-facing errors. Roughly 10–100 lines per day on a single-household install.

`mybibli=debug` Active investigation of an mybibli feature. Adds request-routing, cache hits/misses, scan dispatch decisions. Roughly 10× the info volume.

`mybibli=trace` Deepest one-shot investigation. Includes per-call tracing within mybibli's own code. Very noisy; use briefly and bump back to `info`.

`mybibli=debug,sqlx::query=info` Active investigation AND see every SQL statement (useful when a query is suspect).

How to change the level

Two paths, depending on whether you want the change to survive container restarts or just for one investigation.

From the admin UI (v1.7.1+, recommended)

Log in as Admin, go to `/admin?tab=system`, scroll to the **Logging** section. Enter the new directive in the `Log level` field (anything that `tracing-subscriber::EnvFilter` accepts — plain level or full directive list). Click **Save**. The next log line uses the new filter — no container restart, no `docker compose up -d`. The value is persisted to the `settings` table, so it survives container restarts.

From `docker-compose.yml` / `.env`

Edit the file, set `MYBIBLI_LOG_LEVEL`, then:

```
docker compose up -d
```

The container restarts and the new level takes effect from the next log line. **Caveat:** the admin-saved value in the `settings` table wins on the next boot, so if you've ever saved a level via the

admin UI, that's what will load — the env var only matters on a fresh install before the first save.

12.4 Reading the structured JSON

Logs are emitted as line-delimited JSON. Each line carries:

- `timestamp` — ISO 8601 UTC.
- `level` — `TRACE` / `DEBUG` / `INFO` / `WARN` / `ERROR`.
- `target` — the Rust module that emitted the line (e.g. `mybibli::routes::catalog`). Filter on this to narrow a search to a subsystem.
- `fields.message` — the human-readable summary.
- additional `fields.*` — contextual keys like `user_id`, `volume_id`, `isbn`, `title_id`.

To pretty-print:

```
docker compose exec mybibli tail -n 50 /var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d) \
  | jq -r '[.timestamp, .level, .fields.message] | @tsv'
```

12.5 Sharing a log excerpt for a bug report

When opening an issue at https://github.com/guycorbaz/mybibli/issues:

1. Note the wall-clock time when the bug occurred (e.g. "around 14:30 UTC today").
2. Extract the relevant 5-minute window:

   ```
   # Replace 14:25 / 14:35 with your window.
   docker compose exec mybibli sh -c \
     "awk '/\"timestamp\":\"$(date -u +%Y-%m-%d)T14:2[5-9]/,/\"timestamp\":\"$(date -u
     +%Y-%m-%d)T14:3[0-5]/' \
        /var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d)"
   ```

3. Paste the output into the issue body, fenced as ```` ```json ````.

Redact anything you consider sensitive — log lines may contain borrower names, ISBNs of titles you own, or session-token prefixes (but never full session tokens or password hashes; those are kept out of log emission by design).

12.6 Retention & rotation

- Rotation happens **automatically** at UTC midnight, handled in-process by `tracing-appender::rolling`.
- Files older than **30 days** are purged once per day by a background task. The retention window is hardcoded in v1.7.0; a future release will expose it as an admin setting.
- The current day's file is never deleted (the cutoff is strict).

If you want to keep logs longer than 30 days, copy the rotated files out of the volume to long-term storage:

```
docker compose cp mybibli:/var/log/mybibli/. ./logs-archive/
```

12.7 Resetting a forgotten administrator password

mybibli has no "forgot my password" e-mail flow: a single-tenant household install has no mail server to rely on and no second factor to fall back to. If you are still signed in somewhere, the fix is ordinary — `/admin > Users`, edit your own account, set a new password. Do it *before* that session expires; it is the only thing standing between you and the procedures below.

If you are locked out: the `MYBIBLI_RESET_ADMIN` hatch

Since v1.16.0, the recommended recovery path is a one-shot startup switch. Add to the `mybibli` service's `environment:` block in `docker-compose.yml`, with your admin username as the value:

```
MYBIBLI_RESET_ADMIN=<USER>
```

Restart the container. At startup, mybibli resets that account's password to a freshly generated random string, signs the account out everywhere, writes the new credentials to the log — then **stops on purpose** instead of serving. Read them with:

```
docker compose logs mybibli | grep MYBIBLI_RESET_ADMIN
```

Now remove the variable and restart again; the application refuses to run while it is set, so that a variable forgotten in `docker-compose.yml` cannot leave an account standing with a password that sits in an old log line — every boot would generate a fresh one. Log in with the printed password and change it immediately from `/admin > Users`.

The hatch only accepts an *active administrator* account. An unknown, deactivated, or non-admin username makes startup fail with an explanatory message and changes nothing. Each use is recorded in the audit trail.

Fallback: writing the hash by hand

If you cannot use the hatch (for instance on a version before v1.16.0), the manual procedure below remains valid.

Passwords are stored as Argon2id hashes in PHC string format. The parameters are the `argon2` crate's defaults (`m=19456,t=2,p=1`), and they must match exactly or the new password is rejected at login with no explanation. **Do not generate the hash with a third-party tool** — most default to different parameters. Use the helper shipped with the source, which calls the same function the application uses:

```
git clone https://github.com/guycorbaz/mybibli && cd mybibli
cargo run -q --example hash_password -- 'your new password'
```

It prints one line beginning with `$argon2id$v=19$m=19456`. Apply it on the host running the database, replacing `<HASH>` with that line and `<USER>` with your username:

```
docker compose exec db mariadb -u root -p mybibli \
  -e "UPDATE users SET password_hash = '<HASH>', version = version + 1 \
      WHERE username = '<USER>' AND deleted_at IS NULL;"
```

Then confirm the row before closing the session:

```
docker compose exec db mariadb -u root -p mybibli \
  -e "SELECT username, role, active, deleted_at, \
      LEFT(password_hash,22) FROM users;"
```

Three things that bite

- **Quote the hash with single quotes.** It contains `$` characters. Double quotes let the shell expand them and you will write a truncated value that can never match.
- **Keep `version = version + 1`.** Optimistic locking reads that column; omitting it lets a later edit from the web interface overwrite your change while believing it started from current state.
- **The password reaches your shell history** at both steps. Prefix the commands with a space if your shell is set to ignore such lines, or clear the history afterwards.

This is a direct write to production data with no undo. Run the `SELECT` first: it is reversible only if you recorded the old value.

Reading the login log

The distinction in the log matters when diagnosing: `Login failed: user not found` means no such account (or it is soft-deleted), while `Login failed: invalid password` means the account is healthy and only the password did not match — the case this section addresses. Neither line ever records the attempted password.

Chapter 13

AI agents and proposals

mybibli can be opened to an AI agent — Claude Code, Hermes Agent or any other client of the *Model Context Protocol* (MCP). The agent reads your catalog and suggests improvements: a series and its missing tomes, a missing description or publisher, a cover. Every suggestion arrives as a *proposal* that waits on the **Proposals** page until a librarian accepts, edits or rejects it.

This chapter walks through:

- what an agent can and cannot do,
- creating a *propose* key and allowing the agent's address,
- connecting an MCP client — the generic facts first, then Claude Code,
- the tools the agent sees,
- reviewing proposals: list, detail, comparison, edit, accept, reject,
- expected tomes and the wish list,
- where proposed covers wait, and how they are cleaned up.

13.1 What an agent can and cannot do

Note

An agent never writes the catalog. It reads titles, series and contributors, and it files proposals. Nothing changes in the catalog until a Librarian or an Admin accepts a proposal on the Proposals page. Rejecting, or simply ignoring, a proposal leaves the catalog exactly as it was.

With a *propose* key an agent can:

- search and read titles (bibliographic fields, contributors, volumes with their shelf location, series positions), contributors and series — including the gaps of a closed series;
- list the titles without a cover, or with incomplete metadata;
- propose a new series, or additions to an existing one (titles you hold, and *expected tomes* you do not own yet);

- propose a correction of a title's main title (spelling, punctuation, or the real title of a record that only carries its ISBN), and values for its subtitle, description, publisher, publication year, Dewey code, and contributors to add;
- propose a cover image;
- read the history of its own proposals, with your decisions and your rejection reasons.

It cannot:

- create, change or delete anything in the catalog — titles, volumes, series, contributors, locations, loans;
- see borrowers, loans, users, settings or other keys' proposals;
- erase a value: a proposal only fills or corrects a field, it never empties one, and it only *adds* contributors;
- make the server download anything: a proposed cover is sent as an image, and the web addresses the agent cites as sources are shown to you as links, never fetched by mybibli.

Every proposal must cite its **sources** — the `http` or `https` pages where the agent found the information — so you can check them before deciding.

13.2 Before you start: the network

The MCP endpoint lives at `/api/mcp` on the same web server as the rest of mybibli. It is meant for an agent running on a computer *of your local network*. Like the rest of mybibli (chapter 1), it must never be published on the Internet: the key travels in every request, in clear text on a plain-HTTP LAN.

13.3 Step 1 — create a *propose* key

1. As an Admin, open **Administration** → **API keys** (section 11.2).
2. Give the key a label that names the agent, e.g. *"Claude Code on the office laptop"*. The label is shown next to each proposal the key files.
3. Choose the scope **Read + propose**.
4. Click **Create key** and copy the key from the dialog: it is shown only once. A propose key starts with `mybibli_pr_`.

Only a propose key opens `/api/mcp`; a read-only or read-write key is refused there. Use one key per agent: revoking it (section 13.10) then stops exactly that agent.

13.4 Step 2 — allow the name the agent calls mybibli by

To defend against *DNS rebinding* (a web page tricking a browser into talking to your NAS under a foreign name), the endpoint answers only requests whose `Host` header is on a list. That header carries the name (or address) of the *server* as the agent typed it in its URL — for `http://nas.local:8080/api/mcp`, it is `nas.local:8080`. It says nothing about the machine the agent runs on: the list does not filter clients, every computer of your network calling the same URL sends the same `Host`, and a subnet would make no sense there. Access is controlled by

the *propose* key. Two environment variables, read at start-up, control it (chapter 1, table of section 1.5):

`MYBIBLI_MCP_ALLOWED_HOSTS` Comma-separated host names or addresses, each optionally with a port (`host` or `host:port`; an entry without a port matches any port). Default when unset or empty: `localhost,127.0.0.1,::1` — enough only when the agent runs on the machine that hosts mybibli. A set value *replaces* the default, so on a NAS list every name and address under which agents reach the NAS (its host name, its IP address if an agent uses it), and keep the loopback names if a local client needs them:

```
MYBIBLI_MCP_ALLOWED_HOSTS=nas.local,192.168.1.10,localhost
```

`MYBIBLI_MCP_ALLOWED_ORIGINS` Comma-separated origins (`scheme://host:port`, `:*` for any port). Default: empty. MCP clients such as Claude Code send no `Origin` header and need nothing here; a request that carries an `Origin` is refused unless it is listed. List one only for a client or proxy that forwards one (the MCP Inspector, section 13.5.3). Browser pages cannot call the endpoint at all: it sends no CORS headers.

Set them in `.env` (the shipped `docker-compose.yml` passes both through) and restart the container. These are deployment settings, not Admin settings, on purpose: a page of the web interface must not be able to widen a network defence.

13.5 Step 3 — connect the agent

13.5.1 What any MCP client needs

Whatever the client — Claude Code, Hermes Agent, or another one — it needs these facts:

Setting	Value
Address	`http://<host>:<port>/api/mcp`, where `<host>` is one of the allowed hosts of section 13.4 and `<port>` the port you open mybibli on (`HOST_PORT`, default 8080).
Transport	*Streamable HTTP* (sometimes listed as just *HTTP*). The server is stateless: every call is one `POST` answered with JSON; there is no event stream and no session to keep.
Header	`Authorization: Bearer <propose key>` on every request.
Scope	The key must be a *propose* key (section 13.3). No OAuth, no login.
Protocol	MCP revisions `2025-11-25` and `2026-07-28`. Older revisions are refused.

A client that can set a custom header on a streamable-HTTP server can connect. stdio-only clients and the older "HTTP + SSE" transport are not supported.

13.5.2 Claude Code

Register the server once with `claude mcp add`. Replace the address and the key with yours:

```
claude mcp add --transport http mybibli \
  http://nas.local:8080/api/mcp \
  --header "Authorization: Bearer mybibli_pr_..."
```

The default scope (`-s local`) keeps the server — and its key — in your own Claude Code configuration for the current directory; `-s user` makes it available in every directory. Avoid `-s`

`project`: it writes the key into a `.mcp.json` file meant to be shared with a repository.

Check it with `claude mcp list` (the server should show as connected), or with `/mcp` inside a session. The tools appear to Claude as `mcp__mybibli__<tool>`, e.g. `mcp__mybibli__search_titles`.

Without registering: `--mcp-config`

For a one-off or scripted session, describe the server in a JSON file and pass it on the command line. Keep the file private (it holds the key), e.g. `chmod 600 mybibli-mcp.json`:

```
{
  "mcpServers": {
    "mybibli": {
      "type": "http",
      "url": "http://nas.local:8080/api/mcp",
      "headers": { "Authorization": "Bearer mybibli_pr_..." }
    }
  }
}
```

```
claude --mcp-config mybibli-mcp.json
# non-interactive, mybibli tools only:
claude -p "Which closed series have gaps?" \
  --strict-mcp-config --mcp-config mybibli-mcp.json \
  --allowedTools "mcp__mybibli__list_series"
```

13.5.3 Checking the endpoint with the MCP Inspector

The MCP Inspector is the protocol's reference test client. It is the quickest way to tell a network or key problem from an agent problem. It runs with Node.js (`npx` downloads it on first use) on any machine that can reach mybibli. In the commands below, replace the address and the key.

1. **List the tools** (command-line mode):

   ```
   npx @modelcontextprotocol/inspector --cli \
     http://nas.local:8080/api/mcp --transport http \
     --header "Authorization: Bearer mybibli_pr_..." \
     --method tools/list
   ```

   Expected: the twelve tools of section 13.6.

2. **Call one**:

   ```
   npx @modelcontextprotocol/inspector --cli \
     http://nas.local:8080/api/mcp --transport http \
     --header "Authorization: Bearer mybibli_pr_..." \
     --method tools/call --tool-name get_title --tool-arg id=1
   ```

   Expected: the record of title 1 (or an "unknown title" error if that id does not exist — any answer from the tool proves the connection).

3. **Optionally, the web interface**: run

   ```
   npx @modelcontextprotocol/inspector
   ```

without arguments, open the address it prints, choose the transport *Streamable HTTP*, enter the address, add the `Authorization` header, then *Connect*, *List Tools*, and run `get_title`. The Inspector talks to mybibli through its own local proxy; if that proxy forwards its `Origin` and the connection is refused with *403*, add the origin it shows (typically `http://localhost:6274`) to `MYBIBLI_MCP_ALLOWED_ORIGINS`, restart, and remove it again when you are done.

Each tool call is logged at `info` level as an `MCP tool call` line naming the tool and the key id (chapter 12). The MCP library's own messages are capped at `warn`. To see them while you investigate, enter `info,rmcp=debug` as the **Log level** in **Administration** → **System** → **Logging** (it applies at once, no restart), then put the previous value back. Setting `MYBIBLI_LOG_LEVEL` in `.env` does not help here: the level saved in the panel wins at every start.

13.5.4 When the connection fails

Answer	Likely cause
401	No `Authorization` header, a mistyped key, or a revoked key.
403 `insufficient_scope`	The key is read-only or read-write; create a propose key.
403, other	The `Host` the client uses is not in `MYBIBLI_MCP_ALLOWED_HOSTS`, or the request carries an `Origin` that is not in `MYBIBLI_MCP_ALLOWED_ORIGINS`.
400 / -32022	(The second one is a JSON-RPC error, *Unsupported protocol version.*) The client speaks a protocol revision mybibli does not serve; update the client.
406 / 415	The client does not speak streamable HTTP (wrong `Accept` or `Content-Type`).
405	The client tried `GET` or `DELETE` (an old HTTP + SSE client); choose the streamable-HTTP transport.
413 / 408	A request over ≈ 3.7 MiB (a cover too large) or longer than 30 s.

13.6 The tools

The agent discovers the tools by itself; each carries a description written for it. When it connects, mybibli also tells it which sources it queries automatically: the metadata providers of each media type in their order (the lookup stops at the first one that has the title), and the cover sources tried for books and comics with an ISBN. The agent is asked to search elsewhere first, and told that those sources update their catalogs and may hold a record now. A provider whose key is not configured is not named. For reference:

Reading

`search_titles`
: Free-text search, or an exact ISBN or volume label (V-code); filters by media type and genre.

`get_title`
: One title in full: fields, contributors and roles, volumes and their location, series positions, whether it has a cover, fields edited by hand.

`search_contributors`
: Contributors by name, with how many titles each one has.

`list_series`
: Series by name, with owned positions and gap count; can list only series with gaps.

`get_series`
: One series: its positions, the missing ones of a closed series, its expected tomes.

`list_titles_without_cover`
: Titles with no cover image.

`list_titles_with_incomplete_metadata`
: Titles missing a description, publisher, contributor or publication year.

`list_my_proposals`
: The proposals filed with *this* key: status, decision date, your rejection reason, and when an equivalent proposal may be filed again.

Proposing

`propose_series_create`
: A new series — name, closed (with its number of volumes) or open — with the titles you hold at their positions and expected tomes.

`propose_series_attach`
: Titles to place in an existing series, and/or expected tomes to add to it.

`propose_title_metadata`
: A title correction, subtitle, description, publisher, publication year, Dewey code, and contributors to add, for one title.

`propose_cover`
: A cover image for one title (JPEG, PNG, WebP or GIF, at most 2 MiB).

A tool refuses, with an explanation the agent can act on, a proposal that could never be accepted: an unknown title, a position already taken, a value a librarian's form would refuse, a missing source, a second pending proposal for the same series, or a proposal you rejected recently (section 13.7.8). Each agent may have at most 100 pending cover proposals at a time.

13.7 Reviewing proposals

13.7.1 Seeing that proposals are waiting

Librarians and Admins see a **Proposals** entry in the navigation bar. While proposals are pending it carries a badge with their number (`99+` above 99), and the home page's attention section shows a **Pending proposals** indicator. Both lead to `/proposals`. Anonymous visitors see neither.

13.7.2 The list

`/proposals` shows the pending proposals by default, newest first, 25 per page. Two filters narrow it:

- **Status** — *Pending*, *Accepted*, *Rejected*, *Outdated*, or all;
- **Kind** — *New series*, *Series additions*, *Metadata*, *Cover*, or all.

Each row names the subject — the proposed series name, or the series or title concerned — and **the subject opens the proposal**. A small **View title** (or **View series**) link next to it leads to the catalog page instead. Under the subject, a metadata proposal lists the fields it proposes

(*Proposed: Publisher, Publication year*); next to it, a pending cover proposal shows a thumbnail of the proposed image. The row also gives when it was filed, by which key, its status and its number of lines.

13.7.3 The detail page

The detail page shows who filed the proposal and when, its status, the **Sources** the agent cited (links that open in a new tab — check them), and the **Proposed changes**, one card per line:

- **Series proposals** — the series line (name, type, number of volumes), then each title with its position (and the positions an omnibus covers), then each expected tome with its position, title and ISBN.
- **Metadata proposals** — one comparison table per field, **Current** next to **Proposed** (for contributors: **To add**), side by side even on a phone. A badge in words says whether the field is *New* (empty today), *Changed* or *Unchanged*; the proposed value of a new or changed field is also set off by a thick border, so the highlight never relies on colour alone.
- **Cover proposals** — the current cover (or the placeholder) next to the proposed image, and a sentence that says whether accepting *adds* a cover or *replaces* the current one.

13.7.4 Editing before you accept

While a proposal is pending you can correct it instead of rejecting it. **Edit** opens a line in place; **Save** checks it with the same rules as when the agent filed it.

- A new series: its name, type and number of volumes.
- A title line: its position, the omnibus flag and the last position.
- An expected tome: its position, title and ISBN.
- A metadata field: the proposed value (the current value is the reference and does not change); for contributors, each name and role — clear a name to drop that contributor.

Remove takes a line out of the proposal (title, expected tome, metadata field or contributor line). The line is not applied when you accept, but it stays on record in a *Lines removed before the decision* section. Some lines cannot be removed: the series line, the single line of a cover proposal, and the last line of a metadata proposal — reject the proposal instead. A cover cannot be edited: a different image is a new proposal.

An edited line keeps the agent's value under it (*The assistant proposed: …*) and says who edited it and when. The agent's own history still shows what it filed.

13.7.5 Accepting

Accept opens a confirmation that summarises what will be written — the series created or updated and how many titles and expected tomes, the fields written to the title, the contributors credited, or the cover installed. On confirmation:

- a series proposal creates the series, places the titles and adds the expected tomes; an expected tome whose book the series already holds at another position is skipped and reported;

- a metadata proposal writes the fields and credits the contributors (one already credited in that role is skipped). The written fields count as *edited by hand*: a later automatic metadata download never overwrites them;
- a cover proposal installs the image as the title's cover, also protected from automatic cover downloads.

The success message links to the series or title.

13.7.6 Outdated proposals

A proposal is checked again at the moment you accept it. If its subject changed since the agent filed it — the title or series was edited or deleted, a position was taken in the meantime, a series of the same name now exists, the cover changed, a contributor role was deleted — **nothing is applied** and the proposal becomes *Outdated*; the message says why. Ask the agent for a new proposal based on the catalog as it is now, or make the change by hand.

Only what a proposal touches counts. Accepting a cover does not outdate a pending metadata proposal for the same title, and accepting a description does not outdate a proposal for its publisher: decide them in any order. A metadata proposal becomes outdated when one of *its* fields changed since it was filed (edited by hand, by another accepted proposal, or by a metadata download), a cover proposal when the title's cover changed — and both when the title itself was deleted or re-identified by hand (its name, ISBN or media type corrected; a title correction accepted from a proposal keeps the other proposals valid) or a proposed contributor role was deleted. Editing a field the proposal does not touch outdates nothing.

Revoking a key (section 13.10) also marks all its pending proposals as outdated; the detail page then says they were decided by *the system*.

13.7.7 Rejecting

Reject closes the proposal without changing anything. You may give a **reason** (optional, at most 1000 characters): the agent reads it in its history (`list_my_proposals`) and can learn from it. A proposed cover is deleted when its proposal is rejected.

13.7.8 The resubmission delay

After a rejection, an *equivalent* proposal is refused for a while, so an agent does not file the same thing again and again. Equivalent means the same kind and the same subject: the same series (a new series by its name, ignoring case and accents); for metadata, the same title and *any one* of the rejected fields — a rejected publisher blocks every proposal that includes the publisher, while the year stays open; for a cover, the same title *and the same image* — another image can be proposed at once. The rule applies to every key: creating a new key does not reset it.

The delay is set in **Administration** → **System** → **Proposals** (section 2.5): from 0 (no delay) to 3650 days, default **90 days**. A change applies to the next proposal, without a restart.

13.8 Expected tomes and the wish list

An *expected tome* names a tome of a series that you do not own yet: a position, a title and, if known, an ISBN. Expected tomes come from accepted series proposals, or you enter them yourself.

- **On the series page**, an expected tome fills its gap square with its title, its ISBN and an *Expected* marker. Everyone sees it; the owned / missing counters do not change, since the tome is still missing.
- **Entering one by hand** (Librarian or Admin): **Add expected tome** in the series actions, or **Name this tome** on an empty gap. **Edit** and **Delete** appear under each tome.
- **Add to wish list** puts the tome on your wish list (section 3.10) with its title, its ISBN, and a note naming the series and the position. Clicking twice does not create a second entry.
- **When you catalog the book** — scan its ISBN, or create the title by hand with that ISBN — mybibli recognises the expected tome, places the new title at that position of the series, and the gap disappears. If the book was on your wish list, the usual banner offers to remove it (section 3.10). If the position is meanwhile held by another title, nothing is attached and a warning says what to check.
- **Expected tomes to review** — a tome that no longer fits a gap (the series' number of volumes was lowered below its position, or a title now holds its position) is listed under the grid with the reason. Change its position or delete it.

13.9 Proposed covers: the staging area

A proposed cover is checked like a cover you upload yourself, then reduced to at most 400 px wide (JPEG) — what you review is exactly what acceptance installs. Until you decide, it waits in a hidden `.staging` folder *inside the covers directory* (`COVERS_DIR`; in Docker, `/app/covers/.staging` on the `mybibli_covers` volume). Consequences:

- Nothing to configure: every install that persists its covers also persists the proposed ones across upgrades.
- A proposed image is never public: the `/covers` address refuses hidden folders, and the Proposals page serves it to librarians only, while the proposal is pending.
- Cleanup is automatic. Rejecting a proposal deletes its image; accepting moves it into place; revoking a key deletes the images of its pending proposals. The scheduled purge (section 2.5, *auto-purge interval*) removes anything left behind at its next run — the image of a proposal that went outdated, for instance; a filing that never completed only once it is an hour old.
- Backups of the covers volume (chapter 7) include the folder; restoring it is harmless, the purge sorts it out.

13.10 Revoking an agent's key

In **Administration** → **API keys**, revoke the key. The agent is refused from its next request, all its pending proposals become outdated, and their proposed covers are deleted. Its accepted and rejected proposals stay in the history. To let the agent back in, create a new key and update the client configuration.

Chapter 14

Glossary

Admin A user account with role `admin`. The only role that can open the `/admin` panel and edit users, reference data, trash or system settings.

Anonymous A visitor without a session cookie. Read-only access to the public catalog. No cataloging, no loans, no edits.

Audit The routine "walk the shelves and confirm everything is where the database says it is" workflow described in section 3.8.

BDGest A French-language comic / BD catalogue used as a metadata source for that media type.

BnF *Bibliothèque nationale de France*. The French national library catalogue, used as the primary metadata source for French-language books.

Borrower A person who loans volumes from the library. Modeled as its own entity (name, optional contact information, loan history).

Cataloging The act of registering a new title and / or volume in the database. Typically barcode-driven: scan, confirm, stick label, repeat.

Cover image The visual front of a book / disc / record, downloaded from a metadata provider and cached under `COVERS_DIR` on disk.

CSP Content Security Policy. A browser-enforced instructions header that restricts which scripts and styles the page may execute. mybibli ships a strict policy that forbids inline scripts and styles.

CSRF token A per-session secret embedded into every form. mybibli rejects POST / PUT / DELETE requests whose token does not match — a defense against cross-site request forgery.

Dewey code The Dewey Decimal Classification number, used for topic-based catalog navigation. Optional per title.

EAN-13 The 13-digit barcode standard. ISBNs are encoded as EAN-13 since 2007; CDs, DVDs and other media carry EAN-13 codes too.

Exemplaire Synonym for *volume* (the physical copy). Used by some French libraries; mybibli prefers *volume*.

Expected tome A tome of a series the library does not own yet, named by its position, title and optional ISBN. Shown in its gap on the series page; settled automatically when its ISBN is catalogued (section 13.8).

Genre A free-form classification of a title's subject (Fiction, Non-fiction, Mystery, …). Defined in the genres reference table.

Hard-delete Deleting a row outright. mybibli only hard-deletes from the `/admin > Trash` panel and from the auto-purge background task; everything else is a soft-delete.

HTMX The JavaScript library mybibli uses to swap fragments of the page in place without full reloads. Operators do not need to understand HTMX to use mybibli.

ISBN International Standard Book Number. Identifier printed on the back cover of books, encoded as an EAN-13 barcode.

Last-active-admin guard The rule that prevents deactivating or permanently deleting the last user with the `admin` role. See section 2.6.

L-code A storage-location barcode label. Format: `L` followed by a zero-padded number.

Librarian A user account with role `librarian`. Can catalog, loan, return and edit most data but cannot administer users or settings.

Loan A record that a specific volume is currently out with a specific borrower. Tracks the loan date, due date, return date, and the volume's pre-loan location.

Location The physical place where a volume lives. Modeled as a tree (room → shelf → row), identified by an L-code.

MariaDB The database engine mybibli stores its data in. A fork of MySQL.

MCP Model Context Protocol. The open protocol AI agents use to call tools; mybibli serves it at `/api/mcp` (chapter 13).

Media type The kind of object a title represents: Book, Comic / BD, Audio, Film / series. Drives provider selection and field display.

Multi-position volume A single physical volume that contains several positions in a series — typically an omnibus comic album.

MusicBrainz A volunteer-maintained music metadata catalogue, used as the primary source for audio media.

Node type A label applied to storage locations to indicate what kind of container they are (room, shelf, row, box). Pure label — no semantic enforcement.

OMDb Open Movie Database. Metadata source for films and TV series.

Open Library An open volunteer-driven catalogue of books, used as a metadata source.

Optimistic locking The technique mybibli uses to detect concurrent edits. Two users editing the same title get a clear error on the second save instead of silently overwriting each other.

Propose key An API key with the scope *Read + propose*: it can read the catalog and file proposals, never change the catalog. The only key accepted by `/api/mcp`.

Proposal A change suggested by an AI agent, waiting on the Proposals page until a librarian accepts, edits or rejects it. *Outdated* when its subject changed before the decision.

Reactivation Restoring a soft-deleted row. For users, clears `deleted_at` and lets them log in again. For reference data, the term also covers re-creating a row with the same name as a soft-deleted one, which transparently undoes the delete.

Reference data The four taxonomy tables: genres, contributor roles, volume states, location node types. Editable from `/admin > Reference data`.

Series A named ordering of related titles (e.g. a trilogy or an open-ended comic series). Each title's position in the series is a small integer or a range (for omnibuses).

Setup wizard The four-step first-launch flow at `/setup` that an empty instance presents until an admin account exists.

Soft-delete Setting a row's `deleted_at` column to "now" instead of physically removing the row. Soft-deleted rows are invisible to normal queries but surface in `/admin > Trash` for 30 days, after which the auto-purge hard-deletes them.

Title The bibliographic entity (a book, a film, an album). A title can have multiple volumes — for example, two copies of the same novel on different shelves.

TMDb The Movie Database. A community-curated metadata source for films and TV series.

Trash The `/admin > Trash` tab. Lists soft-deleted entities of every type and lets an admin restore them or permanently delete them.

V-code A per-volume barcode label. Format: `V` followed by a zero-padded number.

Volume The physical object on a shelf — one copy of a title. A title can have many volumes; each carries its own V-code, location and state.

Volume state A reference-data row describing the condition of a volume: New, Good, Worn, Damaged, ….

Index

1.1.0 upgrade, 38

admin panel, 10
admin role, 31, 75
anonymous, 31, 75
`/api/mcp`, 67
audit, 20, 75

backup
  .env, 34
  covers, 34
  database, 34
BDGest, 26, 75
BnF, 26, 75
borrower, 20, 75

Code-128, 12
comic / BD, 24
contributor roles, 13
covers, 75
  .staging, 74
  directory, 4
  persistence, 7
CSRF, 75

Dewey, 50, 75

EAN-13, 12
env vars
  .env.example, 7
  MYBIBLI_MCP_ALLOWED_*, 68
  provider keys, 29
expected tome, 73, 76

film, 24

genres, 13, 76
Google Books, 26

HID keyboard mode, 12

i18n, 14
inactivity timeout, 32
installation, 3

K10plus, 26, 27

L-code, 11, 76
label
  management, 19
labels, 13
last-active-admin guard, 76
librarian, 31, 76
Library of Congress, 26
loan, 76
location, 16, 76
location tree, 10
logs
  directory, 4

MCP, 66, 76
media types, 24, 76
metadata providers, 3, 26
Model Context Protocol, 66
multi-position, 24, 76
MusicBrainz, 24, 26, 76
MYBIBLI_SEED_DEV_USERS, 39

navigation bar, 16
node type, 11, 13, 76

OMDb, 25, 26, 76
omnibus, 24
Open Library, 26, 76

proposal, 66, 77
proposals page, 71
propose key, 67, 76

resubmission delay, 73
roles, 15, 31

scan field, 16
security
  1.1.0 floor, 3
  network exposure, 3

series, 24, 77
session cookie, 32
setup wizard, 6, 9, 77
shelving, 21
soft-delete, 77
storage locations, 10
SYSTEM user, 53

TMDb, 25, 26, 77
troubleshooting
    DATABASE_URL, 52

V-code, 11, 77
valuation, 22
volume, 10, 11, 77
volume state, 13, 16, 77

wish list, 21
/wishlist, 21